

مستند فنی پروژه ۰ نسخه ۱/۳

هم‌رسان

Super App ارتباطی اندروید
پیامک، تماس، مخاطبین و شماره‌گیر

توسعه‌دهنده

علی رودی

طراحی معماری، پیاده‌سازی Flutter و لایه بومی اندروید

Default SMS App

SQLite

BLoC

Kotlin – Android Native

Flutter / Dart

Mirror Sync

Dual SIM

Default Dialer

فهرست مطالب

۱	خلاصه اجرایی	این پروژه چیست و چه مسئله‌ای را حل می‌کند
۲	دامنه محصول و قابلیت‌ها	فهرست کامل امکانات کاربر
۳	آمار پروژه و مقیاس کد	حجم، توزیع و نمودارها
۴	پشته فناوری و وابستگی‌ها	چه چیزی به‌کار رفته و چرا
۵	معماری کلان	لایه‌بندی، مرزها، جریان داده
۶	جریان راه‌اندازی برنامه	از main تا صفحه اصلی
۷	ساختار پوشه‌ها	قرارداد نام‌گذاری و محل افزودن کد
۸	مدیریت وضعیت (BLoC)	۱۴ بلاک سراسری و Guard‌های وضعیت
۹	لایه داده و پایگاه‌داده	۱۷ جدول، نسخه‌اسکیمای ۲۳، Invariant‌ها
۱۰	پل بومی اندروید	۳۱ کلاس Kotlin و کانال‌های ارتباطی
۱۱	نقش Default SMS App و همگام‌سازی آینه‌ای	Mirror Sync – سنگین‌ترین بخش فنی پروژه
۱۲	نقش Default Dialer و رابط تماس	Telecom، صفحه تماس، قفل صفحه
۱۳	موتور جستجو و پردازش متن فارسی	نرمال‌سازی، T9، مرتب‌سازی الفبایی
۱۴	پیام زمان‌بندی‌شده و قالب‌های آماده	تحویل پس‌زمینه و پروتکل فشرده
۱۵	پشتیبانی دو سیم‌کارت	هویت اشتراک در برابر شماره شبیه
۱۶	امنیت و حریم خصوصی	قفل برنامه، رمزنگاری، گزارش خطا
۱۷	کارایی و بهینه‌سازی	مشکلات واقعی و اندازه‌گیری‌ها
۱۸	تست و تضمین کیفیت	۳۸ فایل تست، ۳۹۲ تست
۱۹	دفترچه تصمیم‌های مهندسی	چرا این‌طور و نه آن‌طور
۲۰	محدودیت‌های شناخته‌شده و نقشه راه	آنچه هست، آنچه نیست
۲۱	پیوست – مجوزها و ساخت خروجی	Release و Manifest

این سند برای دو نوع خواننده نوشته شده است

بخش‌های ۱ تا ۳ و ۲۰ بدون پیش‌زمینه فنی قابل‌فهم‌اند و تصویر کلی محصول را می‌دهند. بخش‌های ۵ تا ۱۹ عمق فنی دارند و برای مرور معماری، بازبینی کد و ارزیابی تصمیم‌های مهندسی نوشته شده‌اند. هر بخش فنی با یک پاراگراف «به زبان ساده» شروع می‌شود.

۱. خلاصه اجرایی

مسئله

روی هر گوشی اندرویدی، چهار کار پایه‌ای ارتباطی – پیامک، تماس، تاریخچه تماس و دفترچه مخاطبین – معمولاً در سه یا چهار برنامه جدا انجام می‌شود که هرکدام حافظه، تنظیمات و منطق جستجوی مستقل دارند. برای کاربر فارسی‌زبان، این وضعیت دو مشکل اضافه هم دارد: جستجوی نام‌های فارسی در این برنامه‌ها عملاً کار نمی‌کند (چون «ی» عربی و فارسی، فاصله‌ی مجازی و اعراب را یکسان نمی‌بیند) و مرتب‌سازی الفبایی فارسی نادرست است، چون بر پایه ترتیب کدهای یونیکد انجام می‌شود و حروف پ، چ، ژ، ک و گ در آن ترتیب سر جای خود نیستند.

راه حل

هم‌رسان یک اپلیکیشن اندرویدی است که هر چهار کار را در یک برنامه جمع می‌کند و جای برنامه پیامک و شماره‌گیر پیش‌فرض گوشی را می‌گیرد. برنامه کاملاً فارسی و راست‌به‌چپ است، تقویم شمسی دارد، و موتور جستجوی متن فارسی مخصوص خودش را دارد که مسائل بالا را حل می‌کند. تمام داده روی خود دستگاه می‌ماند؛ پروژه هیچ سرور و هیچ بک‌اندی ندارد.

۱۱۸

کامیت در ۹ ماه

۳۸

فایل تست خودکار

۱۷

جدول پایگاه داده

۲۳۱

فایل منبع

۵۷,۹۴۸

خط کد تولیدی
(Dart + Kotlin)

چه چیزی این پروژه را از یک تمرین Flutter جدا می‌کند

اکثر پروژه‌های Flutter یک لایه رابط کاربری روی یک API هستند. این پروژه API ندارد؛ منبع داده آن خود سیستم عامل اندروید است. برنامه دو نقش سیستمی حساس را می‌گیرد:

- **Default SMS App (ROLE_SMS)** – به محض گرفتن این نقش، اندروید دیگر خودش پیامک‌های ورودی را در پایگاه داده سیستم ذخیره نمی‌کند؛ این وظیفه بر عهده برنامه می‌افتد. یعنی یک باگ در این مسیر برابر است با گم‌شدن پیامک کاربر.
- **Default Dialer (ROLE_DIALER)** – پس از گرفتن این نقش، تنها رابط تماس دستگاه این برنامه است. اگر صفحه تماس بالا نیاید، کاربر هیچ راهی برای جواب دادن به تماس ندارد.

هر دو نقش، خطای «نرم» ندارند. همین موضوع باعث شد بخش بزرگی از مهندسی پروژه صرف مسیرهای شکست، همگام‌سازی دوطرفه با پایگاه داده سیستم، و اجرای درست کد در حالتی شود که پروژه برنامه اصلاً زنده نیست (دریافت پیامک با گوشی خاموش شده، ارسال زمان‌بندی شده، پاسخ سریع از نوتیفیکیشن).

محور	وضعیت
قابلیت‌های اصلی	کامل – پیامک، تماس، مخاطبین، شماره‌گیر، جستجو، زمان‌بندی، قالب، دو سیم‌کارت
پایداری روی دستگاه واقعی	تأییدشده روی Samsung SM-A336E دو سیم‌کارته، اندروید ۱۶
خروجی Release	آماده – امضا، R8/کوچک‌سازی و شناسه نهایی <code>ir.hamrasan.app</code>
پوشش تست	در حال بهبود – لایه BLoC و منطق خالص پوشش دارد؛ تست ابزار (Instrumentation) ندارد
MMS	خارج از دامنه – تصمیم آگاهانه، توضیح در بخش ۲۰
iOS	غیرممکن – iOS اجازه جایگزینی برنامه پیامک/تماس را نمی‌دهد

۲. دامنه محصول و قابلیت‌ها

برنامه یک پوسته چهارتبی است: شماره‌گیر، اخیر (تاریخچه تماس)، مخاطبین و پیام‌ها. جدول زیر همه قابلیت‌های تحویل‌شده را به تفکیک ماژول فهرست می‌کند.

۲/۱ پیام‌ها

قابلیت	توضیح
صندوق ورودی	گفتگو محور، صفحه‌بندی‌شده (Pagination)، با سنجاق‌کردن، بایگانی و شمارش نخوانده‌ها
ارسال و دریافت	مسیر بومی کامل؛ وضعیت تحویل روی حساب پیام: ⌚ در انتظار ← ✓ ارسال ← ✓✓ تحویل / خطا با امکان تلاش مجدد
جستجوی پیام	جستجوی متن کامل روی بدنه همه پیام‌ها (نه فقط پیام‌های بارگذاری‌شده)، با موتور تطبیق فارسی
پیش‌نویس	تخته دوستونی، دسته‌بندی، سنجاق، چندانتخابی
قالب آماده	متن با جای‌گذار [...]؛ فرم پرکردن به صورت خودکار از روی جای‌گذارها ساخته می‌شود
زمان‌بندی ارسال	زمان مشخص، تکرار، «پراکندگی زمان ارسال»، پایان تکرار – با تحویل در پس‌زمینه حتی وقتی برنامه بسته است
کپی کد یکبارمصرف	تشخیص خودکار کد تأیید در پیام دریافتی و نمایش دکمه «کپی»
موقعیت مکانی	درج مختصات جاری به صورت متن در نویسنده پیام (بدون وابستگی به کتابخانه بیرونی)
صفحه‌کلید ایموجی	پنل اختصاصی با دسته‌بندی، «اخیر» و تنوع رنگ پوست
هرزنامه و مسدودسازی	مسدودسازی شماره + گزارش هرزنامه؛ گفتگوی مسدود از صندوق خارج و به صفحه مخصوص منتقل می‌شود
انتخاب و عملیات گروهی	نگه‌داشتن انگشت روی ردیف ← نوار انتخاب چندتایی (حذف، بایگانی، سنجاق، مسدود)
گفتگوی گروهی	یک گفتگوی واقعی با تاریخچه مشترک؛ هر ارسال به صورت پیامک جداگانه برای هر عضو پخش می‌شود (MMS پشتیبانی نمی‌شود، پس پاسخ هر نفر در گفتگوی خودش می‌آید). وضعیت تحویل هر گیرنده جدا دنبال و «ارسال مجدد» فقط برای ناموفق‌ها انجام می‌شود.
صفحه جزئیات گفتگو	لمس سربزرگ گفتگو: نام و شماره مخاطب، تماس، اطلاعات مخاطب، کارت «ارسال با» (انتخاب سیم‌کارت این گفتگو) و بایگانی / مسدودسازی / حذف

۲/۲ تماس و شماره‌گیر

قابلیت	توضیح
صفحه تماس کامل	ورودی و خروجی؛ سکوت، بلندگو، نگه‌داشتن، ادغام تماس، صفحه‌کلید DTMF
نمایش روی قفل صفحه	تماس ورودی روی صفحه قفل نمایش داده می‌شود و صفحه را روشن می‌کند
حسگر نزدیکی	خاموش کردن صفحه هنگام نزدیک شدن به گوش – فقط وقتی خروجی صدا گوشی است، نه بلندگو یا هدست
انتخاب خروجی صدا	فهرست خروجی‌هایی که Telecom واقعاً گزارش می‌کند (گوشی، بلندگو، هدست سیمی، بلوتوث با نام دستگاه)
جستجوی T9	شماره‌گیری با حروف – کلیدهای عددی هم شماره و هم نام فارسی مخاطب را جستجو می‌کنند
شماره‌گیری سریع	نگه‌داشتن کلیدهای ۲ تا ۹؛ * – «+» و ۱ – پست صوتی (با انتخاب سیم‌کارت)
کدهای USSD/MMI	پشتیبانی کامل از *100# و نظایر آن، با مسیر جداگانه از رابط تماس
تاریخچه تماس	همگام‌سازی دوطرفه با تاریخچه سیستم، گروه‌بندی تماس‌های پیاپی، حذف سراسری
تماس بی‌پاسخ	نوتیفیکیشن بومی (وظیفه شماره‌گیر پیش‌فرض)

۲/۳ مخاطبین

قابلیت	توضیح
منبع داده	مستقیم روی دفترچه تلفن دستگاه – بدون کپی محلی، بدون خطر ناهمگامی
مخاطبین سیم‌کارت	خوانده شدن مستقیم از content://icc/adn به‌ازای هر سیم‌کارت فعال
برچسب‌ها	گروه‌های مخاطب قابل ویرایش، با پنهان کردن گروه‌های داخلی سیستم
ادغام تکراری‌ها	تشخیص خودکار (اشتراک شماره یا نام یکسان) و ادغام از راه AggregationExceptions
قالب نام و مرتب‌سازی	«نام / نام خانوادگی» به‌عنوان تنظیم – روی فهرست، جستجو، نوار حروف و تاریخچه تماس همزمان اثر می‌گذارد
برنامه‌های متصل	نمایش اقدام‌های ثبت شده توسط پیام‌رسان‌های دیگر روی همان مخاطب
عکس مخاطب	گرفتن عکس یا انتخاب از گالری با برش دایره‌ای (جابه‌جایی، بزرگ‌نمایی، چرخش)؛ رابط در Dart و پردازش تصویر در Kotlin – بدون کتابخانه بیرونی و بدون درخواست مجوز دوباره
علاقه‌مندی‌ها	جدول مستقل با کلید یکتای نرمال شده

قابلیت	توضیح
قفل برنامه	PIN یا الگو، با ذخیره نمکی و کشیده شده (Salted + Stretched) و کد بازیابی یکبارمصرف
دو سیم کارت	انتخاب سیم در شماره گیری و ارسال، حافظه سیم به ازای هر گفتگو، نشان سیم روی حباب و تاریخچه
تم روشن و تیره	سامانه طراحی مشترک بر پایه نقش سطوح
تقویم شمسی	تبدیل دستی جلالی، بدون وابستگی بیرونی
جستجوی یکپارچه	یک موتور مشترک برای مخاطب، تماس و پیام

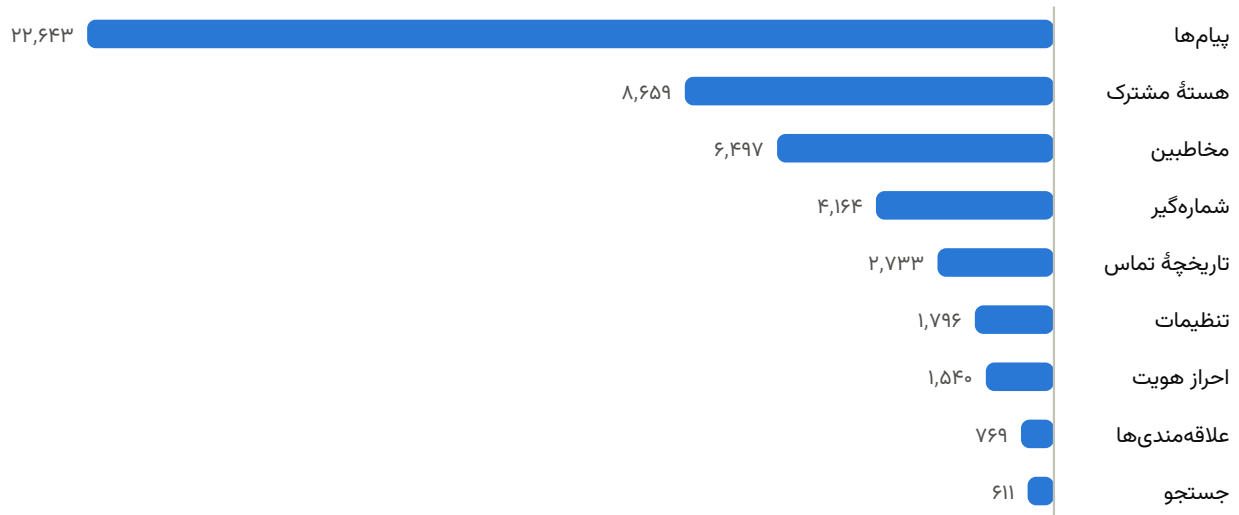
۳. آمار پروژه و مقیاس کد

اعداد زیر مستقیماً از مخزن استخراج شده‌اند و مقیاس واقعی کار را نشان می‌دهند؛ هدف از آوردنشان این است که خواننده بداند با چه اندازه‌ای از کدِ نگهداری‌شونده روبه‌روست.

سنجه	مقدار	توضیح
کد Dart (لایه برنامه)	۴۹,۵۱۰ خط / ۲۰۰ فایل	رابط کاربری، منطق، BLoC، مخازن داده
کد Kotlin (لایه بومی)	۸,۴۳۸ خط / ۳۱ فایل	سرویس‌ها، Receiverها، پل‌های پلتفرم
کد تست	۶,۸۲۰ خط / ۳۸ فایل	Unit، BLoC و ۳۹۲ Widget تست، همه سبز
جدول‌های پایگاه‌داده	۱۷	نسخه اسکیمای: ۲۳ (۲۲ بلوک Migration)
بلاک‌های سراسری	۱۴	همه در یک نقطه فراهم می‌شوند
ماژول‌های ویژگی	۸	ساختار یکسان در همه
مجوزهای اندروید	۲۵	هرکدام با کاربرد مشخص – بخش ۲۱
تاریخچه	۱۱۸ کامیت	آذر ۱۴۰۴ تا شهریور ۱۴۰۵

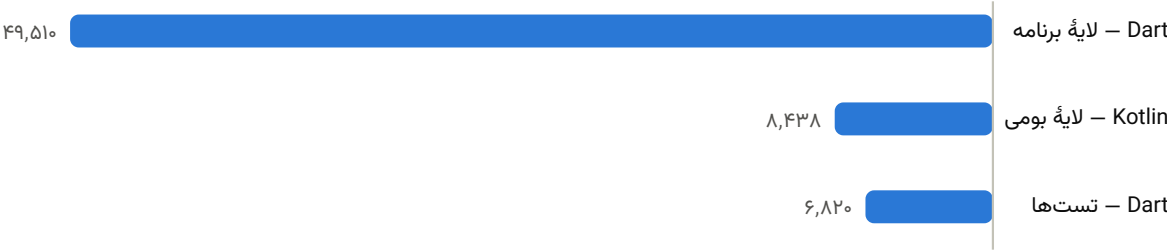
توزیع خطوط کد بر حسب ماژول (Dart)

ماژول «پیام‌ها» به‌تنهایی ۴۶٪ کل کد لایه برنامه است



نمودار ۱ – توزیع خطوط کد میان ماژول‌های ویژگی و هسته مشترک

ترکیب کل مخزن بر حسب زبان و نوع



نسبت تست به کد لایه برنامه: حدود ۱ به ۷ – پوشش روی منطق، نه روی رابط کاربری

نمودار ۲ – ترکیب مخزن؛ حجم کد بومی نشان‌دهنده عمق یکپارچگی با سیستم‌عامل است

۴. پشته فناوری و وابستگی‌ها

سیاست وابستگی پروژه محافظه‌کارانه است: هر کتابخانه باید کاری بکند که نوشتنش به‌دست خودمان گران‌تر از نگهداری وابستگی باشد. سه مورد آگاهانه حذف شدند، که در انتهای همین بخش آمده است.

لایه	انتخاب	دلیل
چارچوب UI	Flutter (Dart SDK ^3.10.1)	یک پایگاه کد، کنترل کامل روی رندر راست‌به‌چپ
زنجیرهٔ ساخت اندروید	Gradle 9.3.1 · AGP 9.1.0 · Kotlin 2.4.0	ارتقا شده در شهریور ۱۴۰۵. همین زنجیره است که هیچ پلاگینی نباید نسخهٔ AGP خودش را روی آن Pin کند – دقیقاً دلیل حذف <code>sentry_flutter</code> در جدول بعدی.
مدیریت وضعیت	<code>equatable</code> + <code>flutter_bloc</code>	رویداد/وضعیت صریح و قابل تست؛ همهٔ گذارها قابل بازپخش در تست‌اند
ذخیره‌سازی	<code>sqflite</code> (SQLite)	تنها موتوری که هم Dart و هم Kotlin می‌توانند روی یک فایل بخوانند
مخاطبین	<code>flutter_contacts</code>	دسترسی مستقیم به <code>ContactsContract</code> ؛ نوشتن بدون کپی محلی
مجوزها	<code>permission_handler</code>	درخواست دسته‌ای پیش از ساخت پوستهٔ اصلی
امنیت	<code>flutter_secure_storage</code> + <code>crypto</code>	ذخیرهٔ نوع قفل در <code>Keystore</code> ؛ هش نمکی ۶۰,۰۰۰ دوری SHA-256
نوتیفیکیشن	<code>flutter_local_notifications</code> + مسیر بومی	نوتیفیکیشن پیامک تماماً بومی است تا با پروسهٔ خاموش هم کار کند
تنظیمات سبک	<code>shared_preferences</code>	پرچم‌های کلید/مقدار
گزارش خطا	<code>sentry</code> (نسخهٔ خالص Dart)	پیش‌فرض خاموش؛ توضیح در بخش ۱۶
فونت	Vazirmatn (۴ وزن)	جایگزین آزاد و نزدیک به IRANYekanX در طرح Figma
تست	<code>mocktail</code> , <code>bloc_test</code> , <code>sqflite_common_ffi</code>	پایگاه‌دادهٔ درون‌حافظه‌ای برای تست واقعی مخازن

وابستگی‌هایی که حذف شدند – و چرا

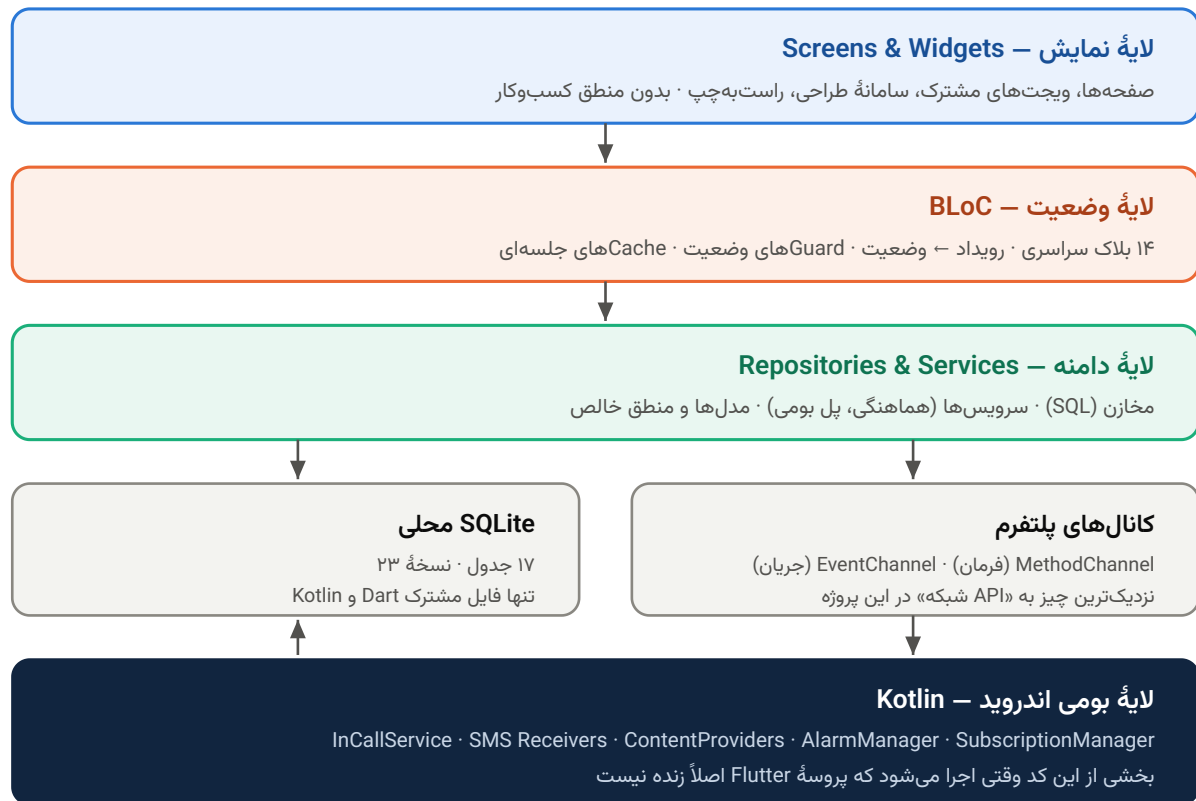
کتابخانه	مشکل و تصمیم
<code>another_telephony</code>	شنوندهٔ سراسری <code>onRequestPermissionsResult</code> آن، نتیجهٔ یک <code>MethodChannel</code> را دوبار پاسخ می‌داد («Reply already submitted») و هر بار که یک فراخوانی تلفنی با دیاپلگ مجوز هم‌پوشانی می‌کرد برنامه کرش می‌کرد. تصمیم: کل دسترسی به <code>Provider</code> پیامک به <code>Kotlin</code> منتقل شد. این حذف، خودش دلیل بخش بزرگی از کد بومی پروژه است.
<code>sentry_flutter</code>	پلاگین <code>Gradle</code> مخصوص خودش را با <code>AGP ۷.۴.۲</code> پین می‌کند و ساخت پروژه را می‌شکند (<code>Could not find com.android.tools.build:gradle:7.4.2</code>). تصمیم: نسخهٔ خالص <code>Dart</code> استفاده شد؛ هزینه‌اش این است که کرش‌های سمت <code>Kotlin</code> گزارش نمی‌شوند.
کتابخانهٔ موقعیت مکانی	یک جفت مختصات با یک ضربه، یک «کتابخانهٔ موقعیت» نیست. تصمیم: استفادهٔ مستقیم از <code>LocationManager</code> پلتفرم در ۱۲۰ خط <code>Kotlin</code> ، بدون افزودن وابستگی جدید به زنجیرهٔ ساخت.
کتابخانهٔ تقویم شمسی	تبدیل جلالی یک تابع ریاضی مشخص است. تصمیم: پیاده‌سازی داخلی (<code>JalaliDate</code>) با تست واحد اختصاصی.

قاعدهٔ کلی که از این تجربه بیرون آمد

در برنامه‌ای که نقش سیستمی می‌گیرد، هر کتابخانه‌ای که به کانال‌های پلتفرم دست می‌زند یک سطح حمله برای باگ‌های همزمانی است. جایی که کتابخانه فقط یک لفاف نازک روی `API` پلتفرم است، نوشتن مستقیم آن لفاف ارزان‌تر و قابل‌پیش‌بینی‌تر است.

۵. معماری کلان

به زبان ساده: برنامه از پنج لایه ساخته شده. هر لایه فقط با لایه زیر خودش حرف می‌زند. صفحه‌ها هیچ‌وقت مستقیماً به پایگاه‌داده یا به اندروید دست نمی‌زنند؛ همیشه از راه یک «بلاک» درخواست می‌دهند و منتظر وضعیت جدید می‌مانند. این قاعده باعث می‌شود بشود هر لایه را جدا تست کرد.



نمودار ۳ – لایه‌بندی؛ فلش دوطرفه سمت راست نشان می‌دهد که لایه بومی هم روی همان فایل SQLite می‌نویسد

قاعده	چرا
هیچ <code>BlocProvider</code> داخل صفحه‌ها ساخته نمی‌شود	همهٔ بلاک‌ها در <code>AppBlocProviders</code> فراهم می‌شوند؛ در غیر این‌صورت باز و بستهٔ یک صفحه، وضعیت گفتگوی باز را دور می‌ریخت
صفحه‌ها هرگز مستقیم به مخزن یا کانال دست نمی‌زنند	هر مسیر داده باید از راه رویداد بگذرد تا در تست قابل بازپخش باشد
شمارهٔ تلفن نرمال‌شده، کلید اتصال سراسری است	پیام، تماس، مخاطب، مسدودی و علاقه‌مندی همه با یک شکل واحد (<code>09xxxxxxxx</code>) به هم وصل می‌شوند
هر تغییر اسکیم فقط با مهاجرت	روی گوشی کاربر پیامک واقعی هست؛ ساخت دوبارهٔ جدول یعنی از دست رفتن داده
هیچ همگام‌سازی، رنگ‌آمیزی فریم را بلوکه نمی‌کند	همگام‌سازی به‌صورت Future جدا اجرا می‌شود و فقط نتیجه‌اش به‌شکل رویداد برمی‌گردد

۵/۲ چهار مرز داده

پروژه لایهٔ شبکه ندارد، اما چهار مرز ورودی/خروجی دارد که هر کدام الگوی تاب‌آوری خودش را دارد:

مرز	چه چیزی از آن می‌آید	حالت شکست و پاسخ ما
SQLite	همهٔ دادهٔ آینده‌شدهٔ برنامه	خطای قفل یا اسکیم ← مهاجرت محافظت‌شده و درج با <code>ConflictAlgorithm.ignore</code>
کانال‌های پلتفرم	پیامک، تماس، سیم‌کارت، موقعیت	کدهای خطای نام‌دار (<code>NO_SIM_CARD</code> ، <code>NO_SERVICE</code> ، ...) و پیام فارسی مخصوص هر کدام
پلاگین‌های دستگاه	مخاطبین، تاریخچهٔ تماس، مجوزها	نبود مجوز باید به «حالت تقلیل‌یافته» منجر شود، نه به کرش
کلید/مقدار	تنظیمات، تم، پرچم‌ها	خواندن همزمان از پیش‌بارگذاری‌شده برای فریم اول

۶. جریان راه اندازی برنامه

به زبان ساده: بین لحظه‌ای که کاربر آیکن را می‌زند تا لحظه‌ای که فهرست پیام‌ها را می‌بیند، پنج دروازه پشت سر هم رد می‌شود: راه اندازی سرویس‌ها، قفل برنامه، مجوزها، درخواست نقش‌های پیش‌فرض، و در آخر پوسته اصلی. ترتیب این دروازه‌ها اتفاقی نیست – هر کدام یک باگ واقعی را حل می‌کند.



نمودار ۴ – زنجیره راه اندازی و دلیل مهندسی هر دروازه

۶/۱ انتخاب تب فرود

برنامه هنگام باز شدن تصمیم می‌گیرد کاربر را روی کدام تب بگذارد: «پیام‌ها» اگر در غیاب کاربر پیامکی رسیده باشد، «اخیر» اگر تماس بی‌پاسخی بوده، و تازه‌ترین این دو اگر هر دو رخ داده باشد. نکته طراحی مهم این است که مبنا «از آخرین باری که کاربر نگاه کرده» است، نه «خوانده نشده»: اگر مبنا خوانده نشده بود، تا وقتی صندوق خالی نمی‌شد هر بار اجرای برنامه تب را می‌ربود. این تصمیم هیچ پرسوجوی اضافه‌ای نمی‌گیرد – از همان وضعیتی که بلاک‌ها به‌هرحال تولید می‌کنند شنیده می‌شود، با یک پنجره مهلت کوتاه، و اگر کاربر از راه نوتیفیکیشن وارد شده باشد تصمیم از پیش قطعی است تا تب زیر دست کاربر عوض نشود.

۶/۲ شروع سرد (Cold Start) – هیچ Spinnerی پیش از برنامه

گزارش کاربر این بود: «هر بار حدود ۲ ثانیه لودینگ». ریشه‌اش یک باگ نبود، سه تصمیم بود که هرکدام جداگانه بی‌ضرر به نظر می‌رسیدند:

آنچه شد	آنچه بود
Gate خالی رندر می‌شود و هر چهار خواندن با <code>Future.wait</code> موازی اجرا می‌شوند. هر کدام در چند فریم جواب می‌دهند؛ یک <code>Spinner</code> آنجا به هیچ‌کس چیزی نمی‌گوید، فقط هر اجرای برنامه را شبیه «در حال بارگذاری» می‌کند – که دقیقاً همان چیزی است که گزارش شد.	یک <code>Spinner</code> <code>DefaultAppGate</code> تمام صفحه روی چهار خواندن پشت سرهم از پلتفرم نگه می‌داشت – در هر بار اجرا
صفحه محلی بلافاصله از <code>SQLite</code> منتشر می‌شود، خواندن دستگاه جدا (<code>Detached</code>) و یک بار در هر بلاک اجرا می‌شود، و «تماس اخیری وجود ندارد» تا وقتی دستگاه واقعاً یک بار خوانده نشده نمایش داده نمی‌شود.	یک <code>CallLogBloc</code> پیش از اولین <code>emit</code> هم منتظر <code>Mirror Sync</code> دستگاه می‌ماند و هم منتظر خواندن کل دفترچه تلفن
این هم جداگانه گزارش شده بود («شماره‌ها نشان داده می‌شوند و با یک فاصله کوتاه اسم‌ها می‌آیند»). حالا اولین <code>emit</code> از جدول <code>contact_name_cache</code> نام‌گذاری می‌شود و خواندن معتبر پشت سرش اجرا می‌شود؛ وقتی چیزی عوض نشده باشد <code>emit</code> دوم با اولی برابر است و چون <code>ThreadsLoaded</code> و <code>CallLogsLoaded</code> هر دو <code>Equatable</code> اند، هیچ بازترسیمی رخ نمی‌دهد.	اولین رسم، ردیف‌ها را با شماره نشان می‌داد و اسم‌ها یک لحظه بعد جا می‌افتادند

اندازه‌گیری روی دستگاه تست

حدود ۱/۸ ثانیه از `am start` تا یک «اخیر» رسم‌شده و پرشده، بدون فریم افتاده – و یک «اخیر» نام‌گذاری‌شده حدود یک ثانیه پس از `am start`. یک نکته مکمل: صندوق خالی در حین اولین درون‌ریزی صراحتاً می‌گوید در حال درون‌ریزی است؛ «هیچ پیامکی موجود نیست» روی گوشی پر از پیامک، شبیه از دست رفتن داده خوانده می‌شود.

۷. ساختار پوشه‌ها

هر ماژول ویژگی دقیقاً یک شکل دارد. این یکنواختی عمدی است: کسی که یک ماژول را بلد باشد، بقیه را هم بلد است و می‌داند کد جدید کجا باید برود.

```
lib/
├─ main.dart          CallUiCoordinator + نقطه ورود
├─ core/              آنچه بیش از یک ویژگی به آن نیاز دارد
│   ├── bloc_providers/ تنها محل ثبت بلاک‌ها
│   ├── constants/    نام جدول‌ها، نسخه اسکیم‌ها - AppConstants
│   ├── database/      مهاجرت‌ها + (تکنمونه) DatabaseHelper
│   ├── navigation/    پوسته ۴ تایی - MainNavigation
│   ├── services/      CrashReporting, DeepLink, Permission, AppLock
│   ├── sim/           مدل/سرویس/بلاک/ویجت دو سیم‌کارت
│   ├── theme/         سامانه طراحی، نقش سطوح، تم روشن/تیره
│   ├── utils/         SearchText, PersianCollator, PhoneNormalizer,
│   │                  DateFormatter (جلالی), ContactNameStyle
│   └─ widgets/        ویجت‌های مشترک بین ویژگی‌ها
└─ features/
    ├── authentication/ قفل برنامه
    ├── call_history/    «اخیر»
    ├── contacts/        دفترچه تلفن
    ├── dialer/          صفحه‌کلید، تماس، سرعت‌شماره‌گیری
    ├── favorites/       علاقه‌مندی‌ها
    ├── messages/        بزرگترین ماژول - ۴۶٪ کد
    ├── search/          جستجوی یکپارچه
    └─ settings/         تنظیمات، مسدودی، هرزنامه
```

هر ماژول:

```
bloc/          رویداد، وضعیت، کلاس بلاک
models/        مدل‌های داده و منطق خالص
repositories/  DatabaseHelper از راه SQLite دسترسی
screens/       (برای اجزای همان صفحه screens/widgets/ + صفحه‌ها)
services/      messages، call_history، contacts، dialer - فقط در
```

قرارداد نام‌گذاری

الگو	مثال
<code><name>_bloc.dart</code> <code>_event</code> / <code>_state</code>	<code>message_bloc.dart</code>
<code><name>_model.dart</code>	<code>scheduled_message_model.dart</code>
<code><name>_repository.dart</code>	<code>message_repository.dart</code>
<code><name>_screen.dart</code>	<code>conversation_screen.dart</code>
<code><name>_service.dart</code>	<code>sms_service.dart</code>

جای افزودن کد جدید

ویژگی جدید ← پوشه تازه زیر `features/` با همان پنج زیرپوشه، و ثبت بلاکش در `AppBlocProviders`. جدول جدید ← ثابت آن در `AppConstants`، بلوک ساخت در `_onCreate` و حتماً یک بلوک مهاجرت در `_onUpgrade`. قابلیت بومی جدید ← یک `Handler` در `Kotlin` و یک سرویس `Dart` که کانال را می پوشاند؛ صفحه ها هیچ وقت مستقیم با `MethodChannel` حرف نمی زنند.

۸. مدیریت وضعیت (BLoC)

به زبان ساده: «بلاک» جعبه‌ای است که فقط دو دهانه دارد – رویداد می‌گیرد و وضعیت بیرون می‌دهد. صفحه‌ها هیچ متغیر مشترکی ندارند؛ هر تغییری روی داده باید به صورت یک رویداد اعلام شود. مزیت عملی این است که هر رفتار برنامه را می‌شود بدون باز کردن رابط کاربری تست کرد.

۸/۱ بلاک‌های سراسری

بلاک	مسئولیت	نکته مهم
ThemeBloc	تم روشن/تیره	در فریم اول بارگذاری می‌شود
AuthBloc	قفل برنامه	کد بازیابی پیش از وضعیت «احرازشده» منتشر می‌شود، وگرنه در همان فریم ساخته و گم می‌شود
SimBloc	فهرست سیم‌کارت‌ها	در راه‌اندازی بارگذاری می‌شود؛ همگام‌سازی تاریخچه تماس منتظر آن می‌ماند
ContactBloc	دفترچه تلفن	به تغییر تنظیم «قالب نام» و به تغییر سیم‌کارت گوش می‌دهد
MessageBloc	صندوق و گفتگو	سنگین‌ترین بلاک؛ Guardهای وضعیت آن در ادامه
DraftBloc	پیش‌نویس و دسته‌بندی	هنگام حذف دسته، فیلتر فعال را هم پاک می‌کند
TemplateBloc	قالب‌های آماده	قالب‌های پیش‌فرض ردیف عادی پایگاه داده‌اند و قابل ویرایش
ScheduledMessageBloc	پیام زمان‌بندی‌شده	هر ۳۰ ثانیه تیک می‌زند و جدول را بازخوانی می‌کند
CallLogBloc	تاریخچه تماس	همگام‌سازی «بی‌صدا» بدون از دست دادن صفحه‌بندی
DialerBloc	صفحه کلید و تماس	در هر فیلتر، فهرست مخاطبین را بازمی‌خواند تا شماره تازه ذخیره شده دیده شود
FavoritesBloc	علاقه‌مندی‌ها	کلید یکتای نرمال شده
SearchBloc	جستجوی یکپارچه	از همان موتور تطبیق مخاطبین استفاده می‌کند، نه یک نسخه موازی
SettingsBloc	تنظیمات	کلیدهای بازنشسته را در اولین بارگذاری از حافظه پاک می‌کند
BlockedNumbersBloc	مسدودی و هرزنامه	مسدودسازی مجدد، «گزارش» را ارتقا می‌دهد و پاک نمی‌کند

۸/۲ Guard های وضعیت – هرکدام یک باگ واقعی

نشانه‌ای که بدون آن دیده می‌شد	Guard
رسیدن یک پیامک در پس‌زمینه، صفحه گفتگوی باز را سفید می‌کرد	<code>LoadThreads</code> وقتی وضعیت فعلی «بارگذاری شده» است، وضعیت «در حال بارگذاری» منتشر نمی‌کند
یک اسکرول سریع، ده‌ها پرس‌وجوی صفحه‌بندی پشت‌سرهم می‌ساخت و چند ثانیه برنامه را قفل می‌کرد	پرچم «در حال اجرا» برای صفحه بعدی (<code>_loadingMoreThreads</code>)
بازگشت از یک گفتگو، فهرست ۲۰۰ ردیفی را به ۵۰ ردیف برمی‌گرداند و اسکرول به بالا می‌پرید	تازه‌سازی نباید فهرست بارگذاری شده را کوچک کند
چون بلاک بین صندوق و گفتگو مشترک است، هر بازگشت از گفتگو اسکرول را از دست می‌داد	حافظه آخرین وضعیت قابل‌رسم صندوق (<code>_lastInbox</code>)
با رسیدن تیک تحویل، نشان سیم‌کارت از روی حباب پیام حذف می‌شد	ساخت مدل فقط با <code>copyWith</code> ، هرگز فیلد به فیلد
بلاک سراسری است و یک لحظه فقط یک گفتگو را نگه می‌دارد؛ گفتگویی که روی گفتگوی دیگر باز می‌شد (لمس شماره داخل پیام، «هدایت») بلاک را روی خودش پارک می‌کرد و صفحه زیرین هنوز Rebuild می‌شد – بازگشت، پیام‌های گفتگوی دوم را زیر سربرگ اولی نشان می‌داد	صفحه گفتگو فقط ردیف‌های <code>thread</code> خودش را رسم می‌کند (<code>_isMine</code>)
حذف، بایگانی، مسدودسازی یا سنجاق روی یک نتیجه جستجو، ردیف را سر جایش می‌گذاشت؛ عمل شبیه بی‌اثر به نظر می‌رسید و تازه با بستن جستجو معلوم می‌شد انجام شده بوده	نتایج جستجو یک Snapshot اند، پس هر تغییر، Query را دوباره اجرا می‌کند
کارگر بومی ردیف را مستقیم در SQLite تمام می‌کرد و «حباب شبخ» تا ری‌استارت می‌ماند	بازخوانی جدول در هر تیک زمان‌بندی، حتی وقتی چیزی ارسال نشده

۸/۳ کش‌های جلسه‌ای

چند کش (Cache) درون بلاک‌ها و سرویس‌ها زندگی می‌کنند تا کار سنگین یک‌بار در هر اجرا انجام شود: نگاشت «شماره ← نام» برای غنی‌سازی صندوق، کش مخاطبین با **Generation Counter**، و کش فهرست سیم‌کارت‌ها با عمر ۳۰ ثانیه.

Generation Counter – یک باگ همزمانی کلاسیک

ذخیره یک مخاطب، کش را باطل می‌کند؛ اما شنونده تغییرات دفترچه تلفن در همان لحظه یک خواندن در جریان دارد. بدون شمارنده، آن عکس قدیمی برنده می‌شد و مخاطبی که از روی یک تماس ذخیره شده بود تا اجرای بعدی برنامه نه در فهرست بود و نه در جستجو. راه‌حل: هر باطل‌سازی شماره را یک واحد بالا می‌برد و خواندنی که پیش از آن شروع شده، حق انتشار نتیجه‌اش را ندارد.

۹. لایه داده و پایگاه داده

به زبان ساده: برنامه یک فایل پایگاه داده SQLite دارد که هم کد Flutter و هم کد اندروید روی همان می‌نویسند. این فایل «آینه»ی داده سیستم است، نه منبع حقیقت آن: منبع حقیقت همچنان پایگاه داده خود اندروید است و بخش ۱۱ توضیح می‌دهد این دو چطور هم‌تراز نگه داشته می‌شوند.

۹/۱ جدول‌ها

جدول	نسخه	نقش
messages	۱	آینه پیام‌ها؛ کلید گفتگو = شماره نرمال شده
call_logs	۱	آینه تاریخچه تماس
contacts	۱	میراثی – چیزی روی آن نمی‌نویسد؛ مخاطبین مستقیم از دستگاه خوانده می‌شوند
favorites	۱	کلید یکتای نرمال شده (بازنویسی شده در نسخه ۱۹)
blocked_numbers	۵	مسدودی + پرچم هرزنامه و زمان گزارش (نسخه ۱۶)
/ archived_threads pinned_threads	۶	وضعیت گفتگو
drafts	۹	پیش‌نویس + سنجاق (نسخه ۱۴)
message_categories	۹	دسته‌بندی پیش‌نویس
message_templates	۱۵	قالب‌های آماده – پیش‌فرض‌ها با شناسه ثابت کاشته می‌شوند
scheduled_messages	۱۰	صف ارسال زمان‌بندی شده با نشانه ادعا (Claim Token)
message_search	۱۸	نمایه FTS5 – ساخته می‌شود ولی روی اندروید غیرفعال است (بخش ۱۳)
thread_sim	۲۰	سیم‌کارت به‌ازای هر گفتگو
speed_dial	۲۱	شماره‌گیری سریع؛ کلید اصلی خود رقم است
message_groups	۲۲	گفتگوی گروهی – شناسه و نام اختیاری
message_group_members	۲۲	اعضا با شماره نرمال شده و نام غیرنرمال شده ذخیره شده
message_group_targets	۲۲	یک ردیف به‌ازای هر گیرنده در هر پیام گروهی – پایه وضعیت تحویل و Tombstone همگام‌سازی
contact_name_cache	۲۳	Cache نام مخاطب برای فریم اول – «شماره نرمال شده ← نام + شناسه مخاطب»

۹/۲ قواعد ثابت اسکیمای (Invariants)

این‌ها قواعدی هستند که شکستنشان باگ‌های خاموش می‌سازد – یعنی چیزی کرش نمی‌کند، فقط نتیجه غلط می‌شود.

پیامد شکستن آن	قاعده
هر همگام‌سازی، پیام‌ها را تکراری می‌کرد	<code>messages</code> نمایهٔ یکتا روی <code>(phone_number, body, timestamp, type)</code> دارد؛ هر درج دسته‌ای باید <code>ConflictAlgorithm.ignore</code> باشد
باگ واقعی و شدید: شماره‌ای که کاربر مسدود می‌کرد به شکل <code>989121234567</code> ذخیره می‌شد، درحالی‌که هر جستجو <code>09121234567</code> می‌پرسید. ردیف موجود بود و هیچ‌وقت پیدا نمی‌شد – یعنی مسدودسازی اصلاً کار نمی‌کرد. نسخهٔ ۱۶ ردیف‌های قدیمی را بازنویسی می‌کند.	ستون <code>normalized</code> در مسدودی و علاقه‌مندی باید دقیقاً همان کلید <code>thread_id</code> باشد
«علامت‌گذاری به‌عنوان نخوانده» روی گفتگویی که فقط پیام فرستاده‌ایم بی‌اثر می‌شد	هر ردیف <i>ارسالی</i> با <code>is_read = 1</code> درج می‌شود؛ شمارش نخوانده‌ها نباید روی <code>type = 'received'</code> فیلتر شود
عمداً چنین انتخاب شد تا با هیچ شماره‌ای تصادم نکند و در عین حال یک <code>thread_id</code> معمولی بماند؛ به همین دلیل صفحه‌بندی، سنجاق، بایگانی، شمارش نخوانده و جستجو بدون هیچ شرط اضافه‌ای روی گروه هم کار می‌کنند. در مقابل، هیچ کدی نباید شناسهٔ گفتگو را بدون بررسی «گروه بودن» به یک تابع نمایشی بدهد.	شناسهٔ گفتگوی گروهی <code>'g:'</code> <code>group_id</code> است، نه یک شمارهٔ تلفن
فهرست شناسه‌های شناخته‌شدهٔ سیستمی باید این جدول را هم شامل شود، وگرنه همگام‌سازی بعدی <code>N-1</code> ردیف را «جدید» می‌دید و یک کپی از پیام گروهی را داخل گفتگوی تک‌نفرهٔ هر عضو وارد می‌کرد.	هر پیام گروهی یک ردیف <code>messages</code> دارد ولی <code>N</code> ردیف در <code>message_group_targets</code>
دلیل وجودش این است که دفترچهٔ تلفن آن طرف یک کانال پلتفرم است و صندوق و «اخیر» هر دو بلافاصله از SQLite رسم می‌شوند؛ بدون آن، فریم اول شماره‌های خالی بود. اگر Upsert می‌بود، مخاطبی که کاربر حذف کرده تا ابد اسمش را روی ردیف‌ها نگه می‌داشت.	<code>contact_name_cache</code> یک <code>Cache</code> است، هرگز منبع حقیقت نیست: فقط یک خواندن کاملِ دستگاه روی آن می‌نویسد، و آن نوشتن کل مجموعه را در یک <code>Transaction</code> جایگزین می‌کند، نه Upsert
حذف سخت تا همگام‌سازی بعدی درست به‌نظر می‌رسید و بعد همه‌چیز برمی‌گشت	حذف محلی باید نرم باشد (Tombstone) با حفظ شناسهٔ سیستمی
یک <code>ADD COLUMN</code> بی‌قید، کل ارتقا را نیمه‌کاره رها می‌کرد	مهاجرتی که جدولی را تغییر می‌دهد که مرحلهٔ قبلی همان ارتقا ساخته است، باید ستون‌ها را بررسی کند

فهرست صندوق ورودی با یک CTE صفحه‌بندی می‌شود که فقط سه ستون سبک حمل می‌کند (شناسه گفتگو، زمان آخرین پیام، سنجاق بودن)؛ کار سنگین – انتخاب آخرین پیام هر گفتگو و شمارش نخوانده‌ها – تنها روی ردیف‌هایی اجرا می‌شود که از LIMIT جان سالم به در برده‌اند. اگر این ساختار مسطح شود، به‌ازای هر پیام کل جدول یک زیرپرسوجوی همبسته اجرا می‌شود.

۱۰. پل بومی اندروید

به زبان ساده: Flutter به تنهایی نمی‌تواند پیامک بگیرد، تماس را جواب بدهد یا از سیم‌کارت مخاطب بخواند. برای این کارها یک لایهٔ Kotlin نوشته شده که با دو نوع کانال به Flutter وصل است: **MethodChannel** برای فرمان‌های تک‌شات (مثل «این پیامک را بفرست») و **EventChannel** برای جریان‌های مداوم (مثل «پیامک تازه رسید» یا «وضعیت تماس عوض شد»).

مسئولیت	فایل
ثبت همه کانال‌ها، نتیجه درخواست نقش‌ها، نمایش روی قفل صفحه	MainActivity.kt
ارسال پیامک با پیش‌بررسی سیم/شبکه، خواندن Provider، درخواست نقش پیامک	SmsHandler.kt
هنگام داشتن نقش: فقط نوشتن پیامک ورودی در پایگاه‌داده سیستم	SmsDeliverReceiver.kt
مسیر «شروع سرد» – وقتی پروسه برنامه اصلاً بالا نیست	IncomingSmsReceiver.kt
تنها تولیدکننده نوتیفیکیشن پیامک؛ رندر قالب‌های فشرده هم اینجاست	SmsNotifier.kt
پاسخ سریع و «خوانده شد» – با برنامه بسته هم کار می‌کند	SmsNotificationActionReceiver.kt
قلب رابط تماس؛ آینه‌کردن وضعیت‌های Telecom به Flutter	CallInCallService.kt
اقدام‌های تماس، USSD، درخواست نقش شماره‌گیر	CallHandler.kt
جریان رویداد تماس؛ وضعیت چسبنده برای موتور تازه‌راه‌افتاده	CallEventStreamHandler.kt
قفل بیداری حسگر نزدیکی – فقط وقتی خروجی صدا گوشی است	ProximityGate.kt
ناظر محتوا روی تاریخچه تماس با ضدلرزش	CallLogSyncHandler.kt
تحویل پیام زمان‌بندی‌شده وقتی برنامه مرده است	ScheduledSmsWorker.kt
زنجیره AlarmManager و بازگرداندن تحویل به Dart وقتی موتور زنده است	/ ScheduledSmsAlarmReceiver.kt Channel / Scheduler
object و نه Handler – چون باید با موتور Flutter مرده هم پاسخ بدهد	SimRegistry.kt
خواندن مخاطبین از content://icc/adn/subId/<id>	SimContactsHandler.kt
ادغام مخاطب با AggregationExceptions	ContactLinkHandler.kt
«برنامه‌های متصل» روی یک مخاطب	ContactExtrasHandler.kt
آینه Kotlin از نرمال‌ساز شماره – اجرای مسدودی در مسیر بومی	BlockedNumbers.kt
آینه Kotlin از رمزگشای قالب فشرده	TemplateWire.kt
یک مختصات با LocationManager پلتفرم	LocationHandler.kt
اجزای اجباری نقش پیامک (MMS عمداً رها می‌شود)	/ HeadlessSmsSendService.kt MmsReceiver.kt
بازچینش هشدارهای زمان‌بندی پس از راه‌اندازی مجدد گوشی	BootReceiver.kt

سه منطق در دو زبان تکرار شده‌اند و باید همزمان تغییر کنند

- نرمال‌سازی شماره – `PhoneNormalizer` (Dart) و `BlockedNumbers` (Kotlin)
 - منطق تکرار، تلاش مجدد و ادعای پیام زمان‌بندی‌شده – `scheduled_message_model.dart` و `ScheduledSmsWorker.kt`
 - پروتکل قالب فشرده – `TemplateWire` در هر دو زبان، کاراکتر به کاراکتر
- دلیل ناگزیر بودنش: این کدها باید در حالتی اجرا شوند که موتور Dart وجود ندارد. خطر واقعی‌اش هم این است که ناهماهنگی با خطا اعلام نمی‌شود – فقط پیامی متفاوت از آنچه فرستنده نوشته رندر می‌شود. به همین دلیل هر سه مورد تست آینه‌ای مشترک دارند.

۱۱. نقش «پیامک پیش‌فرض» و همگام‌سازی آینه‌ای

به زبان ساده: وقتی کاربر این برنامه را به‌عنوان برنامهٔ پیامک پیش‌فرض انتخاب می‌کند، اندروید مسئولیت ذخیرهٔ پیامک‌ها را به آن واگذار می‌کند. از آن لحظه دو انبار داده وجود دارد: انبار خود سیستم (که همهٔ برنامه‌های دیگر می‌بینند) و انبار محلی برنامه. کار سخت این بخش، هم‌تراز نگه‌داشتن این دو در هر شرایطی است – از جمله وقتی پیامکی خارج از برنامه پاک شده باشد.

۱۱/۱ تقسیم مسئولیت هنگام داشتن نقش

مسیر	چه کسی انجام می‌دهد
نوشتن پیامک ورودی در Provider سیستم	SmsDeliverReceiver – و فقط همین
ذخیره در پایگاه‌دادهٔ برنامه + نوتیفیکیشن + به‌روزرسانی رابط	مسیر SMS_RECEIVED مثل قبل
هر ارسال موفق	علاوه بر ارسال، نوشتن در content://sms/sent – وگرنه پیام ارسالی برای هیچ برنامهٔ دیگری دیده نمی‌شود
حذف	همیشه سراسری: اول ردیف Provider، بعد حذف نرم محلی

تلهٔ «دوبار انجام دادن»

اگر هم SmsDeliverReceiver و هم مسیر معمول، پیام را در پایگاه‌دادهٔ برنامه بنویسند، کاربر هر پیامک را دوتا می‌بیند. تقسیم بالا عمداً یک نقطهٔ نوشتن برای هر مقصد تعریف می‌کند.

۱۱/۲ الگوریتم همگام‌سازی آینه‌ای



اجرا: یک‌بار در هر جلسه هنگام بارگذاری صندوق، و بی‌صدا در هر بازگشت به برنامه هرگز روی صف رویداد بلاک نمی‌نشیند – به‌صورت Future جدا اجرا و فقط نتیجه‌اش رویداد می‌شود

نمودار ۵ – سه گام Mirror Sync و دو تصمیم‌گیری پشت آن

سقف ۵۰۰ ردیف حذف شد – و همان سقف، دلیل «پیامک‌های قدیمی نیستند» بود

نسخه قبلی خواندن محتوا را برای هر صندوق به ۵۰۰ ردیف آخر محدود می‌کرد. روی یک گوشی واقعی صندوق ورودی خیلی سریع‌تر از صندوق ارسالی پر می‌شود، پس همان سقف در یک سمت حدود شش هفته عقب می‌رفت و در سمت دیگر چند سال: اسکرول به بالا فقط پیام‌های خود کاربر را نشان می‌داد. حالا سقف برداشته شده است – فهرست شناسه‌ها ارزان است، بدنه پیام فقط برای ردیف‌هایی خوانده می‌شود که واقعاً تازه‌اند، و هر صفحه کنترل را به حلقه رویداد پس می‌دهد. صفحه‌ها پیشرفت را گزارش می‌کنند (با Throttle دو ثانیه‌ای)، پس اولین درون‌ریزی صندوق را در حین کار پر می‌کند، نه در پایان.

۱۱/۳ چرا این همگام‌سازی نباید هیچ‌چیز را بلوکه کند

در نسخه اول، همگام‌سازی داخل خود کنترل‌کننده رویداد `await` می‌شد. نتیجه: حدود ۵ ثانیه Spinner در شروع سرد و بدتر از آن، به‌صاف شدن هر رویداد دیگری پشت آن – یعنی باز کردن یک گفتگو تا پایان همگام‌سازی هیچ واکنشی نداشت. شکل فعلی: صندوق ابتدا از آینه محلی رسم می‌شود، همگام‌سازی جدا اجرا می‌شود و فقط پایانش به‌شکل یک رویداد برمی‌گردد. پیامدهایی که باید حفظ شوند:

- پرچم «انجام شد» فقط در حالت موفق ست می‌شود، تا اجرای اول با مجوز رد شده دوباره تلاش کند.
- شنونده پیامک ورودی پیش از همگام‌سازی راه می‌افتد؛ وگرنه پیامی که وسط آن می‌رسید شنونده‌ای نداشت.
- خطای «مجوز نداریم و محلی هم چیزی نیست» از نتیجه همگام‌سازی می‌آید، نه از بارگذاری.

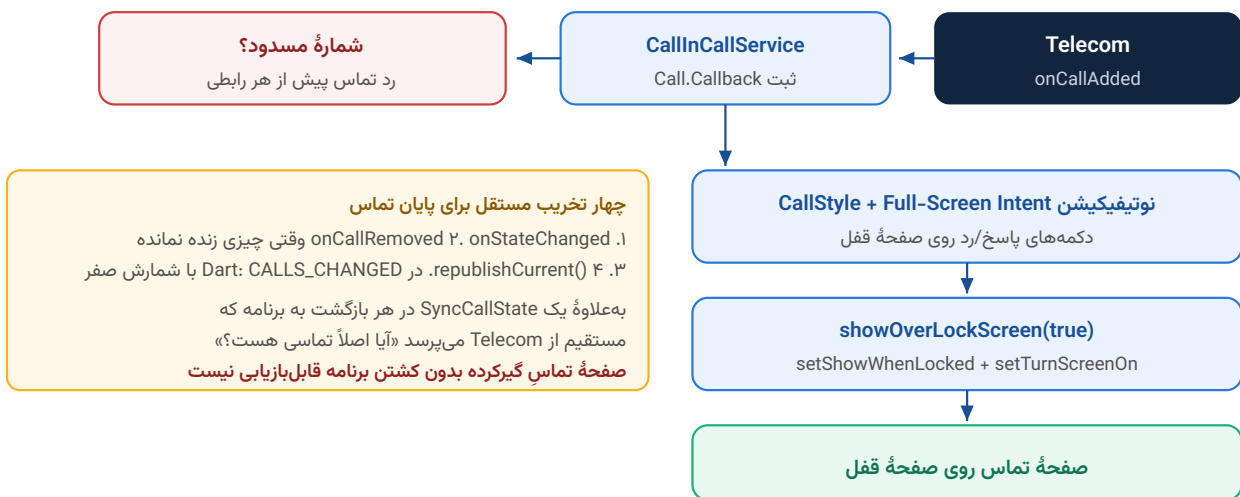
۱۱/۴ نوتیفیکیشن پیامک – یک خط لوله واحد

همه نوتیفیکیشن‌های پیامک ورودی بومی ساخته می‌شوند؛ مسیر Dart برای این کار حذف شده است، چون وجود هر دو یعنی نوتیفیکیشن تکراری. اقدام‌ها (پاسخ درجا با `RemoteInput` و «خوانده شد») نیز کاملاً بومی‌اند و با برنامه بسته کار می‌کنند. نوتیفیکیشن‌ها با شناسه گفتگو برچسب می‌خورند و باز کردن گفتگو، هم نوتیفیکیشن‌های آن را پاک می‌کند و هم «گفتگوی نمایان» را اعلام می‌کند تا تا وقتی صفحه باز است نوتیفیکیشنی ساخته نشود.

۱۲. نقش «شماره‌گیر پیش‌فرض» و رابط تماس

به زبان ساده: با گرفتن این نقش، هر تماس ورودی و خروجی گوشی باید توسط این برنامه نمایش داده شود. اگر صفحه تماس بالا نیاید یا سر جایش نماند، کاربر عملاً گوشی‌اش را از دست می‌دهد. به همین دلیل این بخش بیشترین تعداد «مسیر پشتیبان» را در کل پروژه دارد.

۱۲/۱ زنجیره تماس ورودی



نمودار ۶ – مسیر تماس ورودی و شبکه ایمنی پایان تماس

پدیده	ریشه و پاسخ
صفحه تماس بعد از قطع شدن باقی می ماند	سامسونگ تماس هایی در وضعیت <code>STATE_NEW</code> اضافه می کند – حتی یکی پس از قطع همه. شمردن آنها به عنوان «تماس زنده» باعث می شد <code>onCallRemoved</code> فکر کند تماسی باقی مانده. پاسخ: تعریف صریح مجموعه وضعیت های زنده و نادیده گرفتن <code>NEW</code> (که خودش دوباره اعلام می شود).
تماس رد شده «برقرار» می شود	رفتار اپراتور ایرانسل: پس از رد، تماس گیرنده به پیام اپراتور وصل می شود، پس تلفن واقعاً <code>ALERTING → ACTIVE</code> گزارش می کند. با ردیابی Telecom تأیید شد؛ باگ نیست و نباید «درست» شود.
«ادغام تماس» روی تماس ادغام شده باقی می ماند	Telecom پیوند والد را پس از تحویل میزبان کنفرانس ست می کند؛ پس باید به چهار رویداد جزئیات هم گوش داد، با حذف تکرار – به جز وضعیت «قطع» که همیشه باید فرستاده شود.
USSD کار نمی کرد و به نظر می رسید برنامه * و # را پاک می کند	پاک سازی دستی کاراکترها با <code>[^+0-9]</code> . پاسخ: استفاده از قاعده خود پلتفرم <code>(Uri.fromParts Uri.parse("tel:...") Uri.parse("tel:...") رشته را سر # می برد.</code>
تماس ورودی نامرئی	اگر نوتیفیکیشن برنامه خاموش باشد، هم کارت تماس حذف می شود و هم <code>Full-Screen Intent</code> شلیک نمی کند – و چون نقش شماره گیر دست ماست، هیچ رابط دیگری هم جایگزین نمی شود. پاسخ: ساخت کانال ها در <code>onCreate</code> و پرسیدن صریح این وضعیت در صفحه درخواست نقش.
قطع شدن تماس دوم کل رابط تماس را پایین می کشید	<code>Call Waiting</code> باعث می شود تماس زنگ خورده <code>currentCall</code> شود. پس حالت عادی – تماس گیرنده منتظر قطع می کند در حالی که کاربر هنوز صحبت می کند – به صورت <code>DISCONNECTED</code> برای <code>currentCall</code> می رسید و «تماس تمام شد» منتشر می شد، با کاربری که روی خط بود. پاسخ: <code>liveSuccessorTo</code> رابط را به تماس بازمانده می سپارد (<code>ACTIVE</code> بر <code>Hold</code> و <code>Ringing</code> مقدم است) و فقط یک گوشی خالی <code>DISCONNECTED</code> منتشر می کند.
ماندن یک صفحه «تماس ورودی» مرده روی گوشی بدون تماس	<code>CallUiCoordinator</code> یک فهرست از <code>Route</code> های تماس نگه می دارد، نه یکی. فیلد تکی با <code>Push</code> بعدی بازنویسی می شد و <code>Route</code> قبلی یتیم می ماند: زنگ <code>Call Waiting</code> یک <code>IncomingCallScreen</code> روی <code>InCallScreen</code> زنده <code>Push</code> می کرد و دومی فراموش می شد. حالا هر <code>Route</code> ، <code>Push</code> هایی را که جایگزینشان شده حذف می کند و <code>Teardown</code> همه را پایین می آورد. این رفتار با <code>test/widget/call_ui_coordinator_test.dart</code> قفل شده است.
تماس از شماره ذخیره نشده، اسم نفر قبلی را نشان می داد	<code>copyWith</code> مقدار <code>null</code> را «دست زن» می خواند – که برای هر فیلد دیگری درست است و برای «نام تماس گیرنده» و «سیم کارت تماس» غلط. پس تماسی که <code>PhoneLookup</code> برایش «کسی نیست» جواب می داد، هویت تماس قبلی را به ارث می برد. پاسخ: پرچم های صریح <code>clearActiveSubscriptionId / clearActiveName</code> ، و اعمالشان فقط روی رویدادی که واقعاً شماره حمل می کند. با تست واحد قفل شده است.
تصویر منجمد تماس در سوییچر «برنامه های اخیر»	اندروید <code>Thumbnail</code> یک <code>Task</code> را هنگام <code>onStop</code> می گیرد و تا وقتی برنامه متوقف مانده تازه اش نمی کند؛ پس هر دو راهی که تماس «بیرون از دید» تمام می شود، صفحه تماس را داخل سوییچر

ریشه و پاسخ	پدیده
منجمد می‌کرد. پاسخ: <code>setRecentsSnapshotEnabled(false)</code> برای تمام مدت تماس، و روشن کردن دوباره در <code>onResume</code> فقط وقتی هیچ تماسی باقی نمانده باشد.	
<code>showOverLockScreen(false)</code> از <code>onCreate</code> و <code>onResume</code> هم فراخوانی می‌شود، پس <code>moveTaskToBack(true)</code> بی‌فید هر بار که Activity با Keyguard بالا ساخته یا Resume می‌شد شلیک می‌کرد – برنامه خودش را به پس‌زمینه می‌انداخت. حالا یک Gate اجازه می‌دهد فقط پاک‌کردن پرچمی که یک تماس واقعی ست کرده Task را برگرداند.	«هر بار بعد از قفل‌کردن گوشی، برنامه بسته می‌شود»

یک قاعدهٔ ایمنی که در سند معماری ثبت شده است

هرگز نباید `ROLE_DIALER` درخواست شود مگر آن‌که `CallInCallService` اعلام‌شده و سالم باشد – در غیر این‌صورت تماس‌های ورودی هیچ رابطی نخواهند داشت.

۱۳. موتور جستجو و پردازش متن فارسی

به زبان ساده: جستجوی متن فارسی با `contains` ساده کار نمی‌کند. «محمدرضا» و «محمد رضا»، «علي» با ی عربی و «علی» با ی فارسی، و «۰۹۱۲» با رقم فارسی همه از نظر کاربر یک چیزند و از نظر رشته سه چیز متفاوت. این بخش موتوری است که این تفاوت‌ها را از بین می‌برد – و در کل برنامه فقط همین یک موتور استفاده می‌شود.

۱۳/۱ نرمال‌سازی نام

قاعده	مثال
یکسان‌سازی حروف عربی/فارسی	ي ← ی · ك ← ک · ه ← ه · و ← و
یکسان‌سازی الف	آ / ا / إ / ـا ← ا
حذف اعراب، نیم‌فاصله و نشانه‌های جهت	حرکات، ZWNJ، علائم دوجته
تبدیل ارقام فارسی و عربی به ASCII	۰۹۱۲ / ۰۹۱۲ ← 0912
اختیاری‌بودن فاصله	«محمدرضا» ← «محمد رضا» را پیدا می‌کند

۱۳/۲ تطبیق شماره تلفن

یک شماره در دفترچه تلفن ممکن است به چند شکل ذخیره شده باشد. هم شماره ذخیره شده و هم عبارت تایپ شده به همه شکل‌های هم‌ارز بسط داده می‌شوند: `+98...` `0098...` `98...` `09...` `9...`

یک عدم‌تقارن که باید بماند

ارقام همان‌طور که تایپ شده‌اند در هر جای شماره تطبیق داده می‌شوند، اما شکلی که با حذف صفر یا کد کشور ساخته شده فقط از ابتدای شماره تطبیق می‌خورد. بدون این قید، جستجوی «۰۲۱...» هر موبایلی که «۲۱» در دلش داشت را برمی‌گرداند.

همین هم‌ارزی، یک قاعده رفتاری در صفحه «گفتگوی جدید» می‌سازد: ردیف «ارسال به این شماره» فقط برای شماره‌ای است که به هیچ‌کس تعلق ندارد و وقتی رقم‌های تایپ شده همان شماره یک مخاطب فهرست شده باشند نمایش داده نمی‌شود. آن ردیف رقم‌ها را دقیقاً همان‌طور که تایپ شده‌اند حمل می‌کرد، پس پیام‌دادن به تلفن ثابتی که به صورت «۰۲۱ ۸۸۱۴ ۱۸۵۹» ذخیره شده بود با تایپ همان شماره بدون صفر، یک گفتگوی بی‌نام روی `thread_id` دیگری باز می‌کرد – همان «شماره دوم مخاطب را می‌زنم و اسمش نمی‌آید». آزمون «یک شماره بودن» روی هر دو طرف با همان موتور هم‌ارزی انجام می‌شود؛ یک شماره ناقص عمداً به حساب نمی‌آید، چون تکه‌ای از شماره است نه خود شماره.

۱۳/۳ صفحه‌کلید به‌عنوان جستجوی نام (T9)

همان ارقامی که روی صفحه‌کلید زده می‌شوند دو بار خوانده می‌شوند: یک بار به‌عنوان شماره، یک بار به‌عنوان حروف چاپ شده روی کلیدها. جدول حروف فارسی است (۲ = ا ب پ ت ث ... ۹ = ه ی) و همان جدول روی کلیدها هم چاپ می‌شود – صفحه‌کلیدی که

«کبری» را زیر ۷۲۴ پیدا می‌کند باید این را نشان بدهد، وگرنه قابلیت نامرئی است. تطبیق فقط از ابتدای کلمه و با حداقل طول انجام می‌شود؛ تطبیق میان کلمه با سه رقم، عملاً کل دفترچهٔ تلفن را برمی‌گرداند.

۱۳/۴ مرتب‌سازی الفبایی فارسی

چرا مرتب‌سازی پیش‌فرض غلط است

`String.compareTo` ترتیب UTF-16 است و الفبای فارسی در UTF-16 مرتب نیست – پ، چ، ژ، ک و گ همگی بیرون از بازهٔ ب...ی قرار دارند. نتیجه: نام‌هایی که با این حروف شروع می‌شوند در فهرست پخش می‌شوند و نوار حروف الفبا به جای اشتباه می‌پرد. پاسخ: یک `PersianCollator` اختصاصی که هر حرف را به رتبهٔ درستش نگاشت می‌کند، و کلید مرتب‌سازی یک‌بار به‌ازای هر مخاطب ساخته می‌شود (الگوی `decorate-sort-undecorate`) – ساختن آن داخل تابع مقایسه یعنی نرمال‌سازی هر نام به تعداد $O(\log n)$ بار روی کل دفترچه.

سه چیز دیگر هم به مرتب‌سازی خودمان وابسته‌اند و رایگان به‌دست نمی‌آیند: مرتب‌سازی بر اساس نام خانوادگی (که اصلاً ترتیبی نیست که بشود از Provider خواست)، قرار گرفتن مخاطبین سیم‌کارت سر جای الفبایی خودشان به‌جای انتهای فهرست، و درست پریدن نوار حروف که آفست را از رتبهٔ بخش‌ها انباشته می‌کند.

۱۳/۵ جستجوی پیام – دو مسیر که باید یک جواب بدهند

مسیر	سازوکار و محدودیت
SQL Prefilter + تطبیق نهایی در Dart	Prefilter عمداً فقط می‌تواند بازتر از حد لازم باشد؛ پاسخ نهایی همیشه با تطبیق‌گر Dart داده می‌شود. جدول پیش‌تصویرها در زمان اجرا از خود تابع نرمال‌سازی مشتق می‌شود، نه کپی‌برداری – کپی با گذشت زمان واگرا می‌شد و شروع به تولید «منفی کاذب» می‌کرد.
نمایهٔ FTS5 با توکن‌ساز trigram	تنها توکن‌سازی که FTS5 را به یک نمایهٔ واقعی «زیررشته» تبدیل می‌کند. سه شرط برای استفاده: نمایه وجود داشته باشد، عبارت ≤ 3 کاراکتر باشد، و نمایه عقب‌ماندگی نداشته باشد – چون یک نمایهٔ نیمه‌پر با اطمینان جواب می‌دهد و آنچه هنوز نمایه نشده را بی‌صدا حذف می‌کند.

تصمیم ثبت‌شده: نمایهٔ FTS5 ساخته شد اما روی اندروید غیرفعال است

SQLite سیستم‌عامل اندروید مازول FTS5 را اصلاً ندارد (تأییدشده روی اندروید ۱۶: `no such module: fts5`). فعال‌کردنش یعنی همراه‌کردن SQLite اختصاصی با برنامه. اندازه‌گیری روی صندوق ۵۰ هزارتایی: اسکن معمولی یک عبارت نادر را در ۶۸ میلی‌ثانیه و نمایه در ۱ میلی‌ثانیه جواب می‌دهد – اما اسکن یک عبارت پرتکرار را در ۷ میلی‌ثانیه جواب می‌دهد و پرکردن اولیهٔ نمایه حدود ۱۰ ثانیه روی گوشی طول می‌کشد. در مقابل، کد Kotlin همین فایل را در حالتی باز می‌کند که پروسهٔ برنامه مرده است (پیامک شروع سرد، کارگر زمان‌بندی، پاسخ از نوتیفیکیشن) – دو موتور SQLite باید سر WAL و قفل‌گذاری توافق کنند و یک خطای `database is locked` آنجا یعنی پیامک از دست‌رفته. نتیجه: رد شد. بازگشایی این تصمیم فقط با اندازه‌گیری روی صندوق بزرگ واقعی و با پیش‌شرط داشتن تست همزمانی Dart/Kotlin روی یک فایل.

۱۴. پیام زمان‌بندی‌شده و قالب‌های آماده

۱۴/۱ دو تحویل‌دهنده، یک صف

به زبان ساده: پیام زمان‌بندی‌شده باید سر وقت برود، حتی اگر کاربر برنامه را بسته باشد. پس دو مسیر تحویل وجود دارد: یکی در Dart وقتی برنامه باز است، یکی در Kotlin وقتی نیست. خطر بدیهی این است که هر دو یک پیام را بفرستند.

تحویل‌دهنده	زمان اجرا
Dart — <code>ScheduledMessageBloc</code>	تیک هر ۳۰ ثانیه در حالی که برنامه زنده است
Kotlin — <code>ScheduledSmsWorker</code>	از یک هشدار <code>AlarmManager</code> وقتی پروسه مرده است

هماهنگی: «ادعا» پیش از ارسال

یک `UPDATE ... SET status='sending', claim_token=?` اتمی، ردیف‌های سررسیده را مهر می‌زند و هر تحویل‌دهنده فقط ردیف‌هایی را پردازش می‌کند که نشانهٔ خودش را دارند. هیچ پیام زمان‌بندی‌شده‌ای بدون ادعای قبلی ارسال نمی‌شود. ردیفی که بیش از حد در وضعیت «در حال ارسال» بماند، دوباره آزاد می‌شود.

یک ظرافت دیگر: وقتی هشدار شلیک می‌شود و موتور Flutter زنده است، تحویل عمداً به Dart برگردانده می‌شود. دلیل: کارگر بومی مستقیم روی SQLite می‌نویسد، پس یک ارسال بومی در حالی که برنامه باز است، وضعیت بلاک‌ها را کهنه می‌گذارد — گفتگو «حباب شبیخ» خود را نگه می‌داشت و پیام ارسال‌شده را هرگز نشان نمی‌داد.

پراکندگی زمان ارسال (Jitter)

Jitter یک ویژگی تزئینی نیست: از سه‌گانهٔ (شناسه، زمان زمان‌بندی، شمارهٔ تکرار) به‌صورت قطعی مشتق می‌شود. اگر در هر تیک دوباره قرعه‌کشی می‌شد، یک ردیف می‌توانست زودتر از موعد شلیک کند. چون این پنجره در SQL قابل‌بیان نیست، ردیف‌های سررسیده خوانده، در حافظه فیلتر و فقط بازماندگان ادعا می‌شوند؛ نسخهٔ Kotlin هم دقیقاً همین دروازه را اعمال می‌کند.

شرط لازم این است که هر سه طرف عدد یکسانی حساب کنند — و نمی‌کردند: Dart با `Object.hash`، Kotlin با `hashCode` خودش، و زمان‌بند هشدار با یک `Random` تازه. نتیجه این بود که هشدار گوشی را در لحظه‌ای بیدار می‌کرد که تحویل‌دهنده آن را «هنوز باز نشده» می‌دید، ردیف به حالت انتظار برمی‌گشت و پیام تا بیدارباش بعدی معطل می‌ماند. حالا یک `FNV-1a` سی‌و‌دوبیتی مشترک به‌کار می‌رود که در تست با بردارهای مرجع قفل شده است، و زمان‌بند دقیقاً همان لحظهٔ مؤثر را هدف می‌گیرد.

چرا پیام‌ها دیر می‌رفتند — و سه اصلاحی که لازم بود

در آزمون میدانی، پیامی که برای ساعت ۹ صبح زمان‌بندی شده بود ساعت ۱۱ — و دقیقاً در لحظهٔ باز شدن برنامه — ارسال شد. ریشه در جای غیرمنتظره‌ای بود:

مسئله	اصلاح
بلاک زمان‌بندی تنبل ساخته می‌شد	<code>BlocProvider</code> به‌صورت پیش‌فرض تنبل است و این بلاک فقط از داخل صفحه گفتگو خوانده می‌شد؛ پس در یک اجرای عادی اصلاً ساخته نمی‌شد: نه تیک ۳۰ ثانیه‌ای، نه هیچ بازتنظیمی برای هشدار پیام‌های در انتظار. پیام همان‌جا می‌ماند تا کاربر تصادفاً یک گفتگو باز کند و اولین جاروب آن را «سررسیده» ببیند. حالا <code>lazy: false</code> ، به‌علاوه بازتنظیم هشدار در هر بار بازگشت به برنامه.
هشدار در برابر «حالت آماده‌باش» مقاوم نبود	<code>setExactAndAllowWhileIdle</code> از Doze عبور می‌کند ولی از App Standby نه؛ برنامه‌ای که کاربر امروز بازش نکرده، هشدار دقیقش به چند ساعت یک‌بار محدود می‌شود. برای پیام‌هایی که تا ۲۴ ساعت آینده‌اند حالا <code>setAlarmClock</code> استفاده می‌شود — تنها نوع هشدار که سیستم آن را به تعویق نمی‌اندازد. بهایش نمایش نشان «ساعت زنگ» سیستم است.
هشدار و دروازه تحویل یک لحظه را نام نمی‌بردند	همان مسئله Jitter که بالاتر توضیح داده شد.

تأیید روی دستگاه

پیام برای ساعت ۱۶:۴۰ زمان‌بندی و پروسه برنامه پیش از آن کشته شد. کارگر بومی رأس 16:40:00.149 ردیف را ادعا کرد، پیامک را فرستاد و هشدار بعدی را خودش تنظیم کرد — بدون هیچ تأخیری. خروجی `dumpsys alarm` این هشدار را به‌عنوان «نخستین بیدارباش از حالت خواب» گوشی فهرست می‌کند.

۱۴/۲ قالب آماده و پروتکل فشرده روی سیم

به زبان ساده: یک پیام قالبی («جلسه در تاریخ ... ساعت ... در محل ... برگزار می‌شود») عمدتاً متن ثابتی است که گیرنده هم دارد. پیامک فارسی در هر بخش فقط ۷۰ کاراکتر جا می‌دهد. پس برای قالب‌های از پیش تعریف‌شده، به‌جای فرستادن متن کامل، فقط یک سربرگ کوتاه و پاسخ‌های کاربر فرستاده می‌شود و طرف مقابل متن را بازسازی می‌کند.

علی|جلسه هفتگی|۱۴۰۵/۰۵/۲۰|۱۰:۳۰|اتاق ۳ → 49 کاراکتر[#T1:mtg:1]
 علی عزیز|جلسه جلسه هفتگی در مورخه ۱۴۰۵/۰۵/۲۰ ساعت ۱۰:۳۰
 در محل اتاق ۳ برقرار می‌باشد. → 85 کاراکتر

دلیل	قاعده
دو گوشی ردیف‌های یکسانی ندارند. اگر شناسهٔ ردیف حمل می‌شد، ویرایش یک قالب در یک گوشی پیام‌های ورودی دیگر را بی‌صدا بازنویسی می‌کرد.	سربرگ به ثابت‌های کامپایل‌شده اشاره می‌کند، نه به ردیف پایگاه‌داده
بک‌اندی وجود ندارد، پس طرف مقابل نمی‌تواند متنی را که کاربر اختراع کرده بداند.	قالب‌های ساختهٔ کاربر عمداً خارج از این پروتکل‌اند
Provider سیستم هم همین را دارد و همگام‌سازی آینه‌ای با همین تطبیق می‌دهد؛ رمزگشایی فقط در لحظهٔ نمایش انجام می‌شود.	پایگاه‌داده همان چیزی که روی سیم رفته را ذخیره می‌کند
همهٔ نوتیفیکیشن‌های پیامک بومی‌اند؛ بدون این آینه، نوار اعلان متن رمزی را نشان می‌داد و گفتگو متن واقعی را.	Kotlin نیمهٔ «دریافت» را آینه می‌کند
گیرنده‌ای که این برنامه را ندارد نباید یک رشتهٔ نامفهوم دریافت کند مگر آن‌که واقعاً صرفه‌جویی‌ای در کار باشد.	اگر فشرده‌سازی واقعاً کوتاه‌تر نشود، متن ساده می‌رود

این فهرست فیلدهای بدون‌متن ثابت، همان درزی است که فاز بعدی پروژه – پیامک رمزنگاری‌شده – در آن می‌نشیند.

۱۵. پشتیبانی دو سیم کارت

به زبان ساده: روی گوشی دو سیم کارته باید معلوم باشد پیام با کدام سیم رفته، تماس با کدام برقرار شده، و پیام بعدی با کدام برود. روی گوشی تک سیم، هیچ کدام از این ها نباید دیده شود.

اصل طراحی	توضیح
یک سیم کارت ← هیچ تغییری	هر عنصر رابط کاربری مربوط به سیم، به یک پرچم «چندسیمی» وابسته است؛ گوشی تک سیم باید دقیقاً مثل قبل از وجود این قابلیت به نظر برسد.
هویتی که سفر می کند subscriptionId است، شماره شیار فقط نمایش داده می شود	شناسه اشتراک یک شمارنده مبهم و مخصوص همان دستگاه است که با درآوردن و گذاشتن کارت عوض می شود؛ پس تنظیمات ذخیره شده «اشتراک» را نام می برد و رابط کاربری «سیم ۱» را.
NULL یعنی «نامشخص»، نه «سیم ۱»	ردیف های قدیمی، ردیف هایی که سازنده گوشی مهر نرده و تماس های VoIP همه NULL اند. نشان دادن سیم اشتباه بدتر از نشان ندادن آن است.
پیامک، سیم را به ازای هر گفتگو به خاطر می سپارد	و در لحظه ارسال ثبت می شود نه در لحظه انتخاب – چون آنچه اهمیت دارد این است که واقعاً چه رفت. یک «سیم انتخاب شده سراسری» برای همه گفتگوها جز آخری غلط می بود.
انتخابگر فقط وقتی ظاهر می شود که انتخاب واقعی باشد	دو سیم و نبود پیش فرض سیستمی. دوباره حدس زدن تنظیم اندروید کاربر خودش باگ است.
...اما همیشه یک راه بازگذاری صریح هست	وگرنه یک پیش فرض پین شده، کارت دیگر را از داخل برنامه غیرقابل دسترس می کرد – که دقیقاً اشتباه نسخه اول بود. دو مسیر: نگاه داشتن هر دکمه تماس، و ردیف صریح «تماس با سیم ...» در شیت های تماس.
هیچ «تراشه سیم» روی صفحه نیست	نه در نویسنده پیام، نه کنار دکمه تماس در شماره گیر – همان کاری که Google Messages و Google Phone می کنند. اینکه یک گفتگو با کدام کارت ارسال می شود، ویژگی خود گفتگوست و یک بار پرسیده می شود؛ تراشه ای که میان دکمه «+» و متن راهنما گیر کرده بود، بیشتر شبیه تزئین نویسنده به نظر می رسید تا پاسخ به آن پرسش. جای آن حالا کارت «ارسال با» در صفحه جزئیات گفتگو است (لمس سربرگ گفتگو).
کارت «ارسال با» همیشه یک کارت مشخص را نام می برد	به ترتیب: سیمی که این گفتگو قبلاً با آن فرستاده ← پیش فرض پیامک خود اندروید ← شیار ۱. صفحه ای که کارش نوشتن پاسخ به «این پیام با کدام کارت می رود» است نمی تواند «هیچ کدام» بنویسد، پس یک کارت قطعی resolve می شود و ارسال هم دقیقاً با همان انجام می گیرد.
پیش بررسی ارسال باید به ازای همان سیم باشد	خواندن وضعیت شبکه اشتراک پیش فرض باعث می شد دو کارت درباره هم دروغ بگویند: ارسال روی سیم ۲ رد می شد چون سیم ۱ آنتن نداشت، و ارسال روی کارتی که واقعاً آنتن نداشت از بررسی رد می شد.

Telecom یک «حساب تلفن» را نام می‌برد، نه یک اشتراک – و هیچ API عمومی این دو را به هم وصل نمی‌کند

نگاشت با یک زنجیره پشتیبان انجام می‌شود: برابری شناسه دستگیره با شناسه اشتراک (اندروید مدرن) ← شناسه ICC ← برچسب حساب ← ترتیب شمار. همین نگاشت هم برای تماس خروجی و هم برای تاریخچه تماس به کار می‌رود. چون نتیجه این نگاشت ذخیره می‌شود، اجرای آن پیش از آماده شدن فهرست سیم‌ها، برای همیشه روی همه تماس‌ها مهر «بدون سیم» می‌زد – پس همگام‌سازی تاریخچه صریحاً منتظر آن می‌ماند.

گذاشتن سیم دوم در حال اجرا

همه عناصر مربوط به سیم، فهرست سیم‌ها را ایستا و همگام درون `build` می‌خوانند – یک حباب پیام یا یک ردیف تاریخچه نمی‌تواند منتظر کانال پلتفرم بماند. این تصمیم درست است و در عین حال دقیقاً دلیل آن بود که گذاشتن سیم دوم تا بسته شدن و باز شدن کامل برنامه هیچ اثری نداشت: هیچ ویجتی به فهرست سیم‌ها وابسته نبود. راه حل یک شمارنده نسخه روی همان سرویس است که با هر تغییر فهرست یا تغییر پیش فرض‌ها بالا می‌رود، و یک ویجت پوششی که آن را به بازترسیم تبدیل می‌کند.

نکته دوم: رویداد تغییر فهرست، داده‌های مشتق شده را با خود حمل نمی‌کند. پیش فرض‌های سیستم و نگاشت «حساب تلفن» ← اشتراک» هم باید دوباره خوانده شوند؛ وگرنه فهرست زنده به روز می‌شد ولی پیش فرض‌ها همچنان می‌گفتند «تک سیم» و نگاشت فقط یک ورودی داشت – یعنی انتخابگر چیزی برای انتخاب نداشت و هر تماس جدید «بدون سیم» ثبت می‌شد.

مخاطبین سیم کارت – یک کشف غیرمنتظره

مخاطبین سیم کارت در `ContactsContract` نیستند. سیستم آن‌ها را فقط زمانی وارد می‌کند که برنامه مخاطبین پیش فرض دستگاه یک حساب سیم ثبت کرده باشد – که روی گوشی‌ای که این برنامه سطح مخاطبین آن است، اتفاق نمی‌افتد. نتیجه: کتابخانه مخاطبین با سیم دوم دقیقاً همان فهرستی را برمی‌گرداند که بدون آن. پاسخ: خواندن مستقیم `content://icc/adn/subId/<subId>` به ازای هر اشتراک فعال – و نه آدرس خالی `content://icc/adn`، که همیشه به اشتراک پیش فرض اشاره می‌کند و مخاطبین سیم ۱ را دو بار نشان می‌داد.

۱۶. امنیت و حریم خصوصی

این برنامه پیامک‌های کاربر، دفترچه تلفن و تاریخچه تماس او را در اختیار دارد. سیاست پروژه ساده است: هیچ داده کاربر از دستگاه خارج نمی‌شود.

۱۶/۱ قفل برنامه

موضوع	پیاده‌سازی
ذخیره اعتبارنامه	نه به صورت متن ساده: قالب <code>v2:<salt>:<hash></code> با ۶۰,۰۰۰ دور SHA-256. ردیف‌های نسخه‌های قدیمی متن ساده‌اند و در <i>اولین اعتبارسنجی موفق</i> ارتقا می‌یابند – هرگز در یک حدس اشتباه.
نوع قفل	در <code>flutter_secure_storage</code> (پشتوانه Keystore اندروید)
فراموشی رمز	هنگام تعیین PIN یک کد بازیابی یکبارمصرف ساخته و فقط یک بار نمایش داده می‌شود (الفبای بدون ابهام، بدون 0/0 و 1/1؛ فقط هش ذخیره می‌شود). این کد هرگز مستقیماً قفل را باز نمی‌کند، بلکه به مسیر «تعیین رمز جدید» می‌رود – کدی که روی کاغذ نوشته می‌شود نباید به رمز دوم تبدیل شود.
قفل مجدد	وابسته به چرخه عمر برنامه؛ صفحه تماس عمداً بالاتر از قفل قرار دارد و بقیه برنامه قفل می‌ماند

۱۶/۲ گزارش خطا – پیش‌فرض خاموش

گزارش خطا تنها وقتی فعال می‌شود که هم یک DSN در زمان ساخت تزریق شده باشد و هم کاربر صراحتاً آن را روشن کرده باشد (پیش‌فرض: خاموش). حتی در آن حالت:

- `sendDefaultPii` خاموش؛ بدون اسکرین‌شات و بدون سلسله‌مراتب ویجت.
- ردپاهای `console`، `http` و `query` کاملاً حذف می‌شوند – چون `debugPrint` در این برنامه به‌طور مشروع متن پیام‌ها را چاپ می‌کند.
- هر رشته باقی‌مانده از یک پالایشگر عبور می‌کند که هر دنباله ۴ رقم یا بیشتر را جایگزین می‌کند.

هیچ‌کدام از این قیدها نباید شل شود

این برنامه پیامک پیش‌فرض دستگاه است. یک ردپای لاگ حاوی متن پیام، برابر است با نشت محتوای خصوصی کاربر به یک سرویس بیرونی.

تصمیم	دلیل
پشتیبان‌گیری خودکار خاموش	پایگاه دادهٔ پیامک متن ساده است؛ <code>allowBackup=false</code> به‌علاوهٔ قواعد استخراج داده
مسدودسازی در هر دو مسیر دریافت اجرا می‌شود	مسیر Dart برای وقتی برنامه زنده است، مسیر Kotlin برای وقتی نیست
مختصات مکانی ذخیره یا ردیابی نمی‌شود	فقط داخل همان پیامکی که کاربر انتخاب می‌کند از دستگاه خارج می‌شود
R8/کوچک‌سازی در خروجی Release	کاهش حجم و مبهم‌سازی پایه
هیچ سرور، هیچ حساب کاربری، هیچ تله‌متری	پروژه لایهٔ شبکه ندارد – این را می‌شود با یک نگاه به وابستگی‌ها تأیید کرد

۱۷. کارایی و بهینه‌سازی

همهٔ موارد این بخش، مشکلات مشاهده‌شده روی دستگاه واقعی بودند، نه بهینه‌سازی حدسی. هرکدام یک نشانهٔ قابل‌توصیف داشت.

نشانه	ریشه	راه حل
حدود ۵ ثانیه Spinner در باز کردن اولیه تب پیام‌ها	<code>await</code> کردن همگام‌سازی داخل کنترل‌کننده رویداد	رسم از آینده محلی + اجرای همگام‌سازی به‌صورت Future جدا
مکث چندثانیه‌ای هنگام اسکرول سریع صندوق	هر اعلان اسکرول یک رویداد «صفحه بعد» می‌ساخت؛ <code>Guard hasMore</code> فقط بعد از بازگشت صفحه قبلی برمی‌گشت	پرچم «در حال اجرا» جدا از <code>hasMore</code>
قفل‌شدن رابط وسط اسکرول روی گوشی پر	کشیدن همه ردیف‌های محلی از کانال پلتفرم Dart برای تفریق در	تفريق داخل SQLite با جدول موقت شناسه‌ها
ارقام هنگام تایپ سریع روی صفحه‌کلید گم می‌شدند	صفحه‌کلید داخل یک شیت کشویی است؛ تشخیص‌دهنده ضربه باید در «Gesture Arena» با کشیدن شیت رقابت می‌کرد و می‌باخت	استفاده از <code>Listener</code> خام و شلیک روی لمس، نه روی ضربه – یک <code>Listener</code> عضو Gesture Arena نیست
تأخیر در واکنش کلیدها هنگام شماره‌گیری	یک <code>BlocBuilder</code> در بالای صفحه شماره‌گیر، هر فشار کلید ۱۲ کلید متحرک و فهرست پیشنهادها را بازسازی می‌کرد	حذف کامل آن؛ فقط سه ناحیه باریک و مشروط بازسازی می‌شوند
کل صفحه برای مدت انیمیشن سفید می‌شد	<code>clamp(220, 130)</code> – کران پایین بزرگ‌تر از کران بالا – که در هر بیلد نویسنده پیام محاسبه می‌شد	حذف <code>clamp</code> و ثبت ارتفاع صفحه‌کلید فقط در حالت «رو به بالا»
«هر بار ۲ ثانیه لودینگ» در باز کردن برنامه	یک Spinner تمام‌صفحه روی چهار خواندن پشت‌سرهم پلتفرم، به‌علاوه <code>await</code> شدن Mirror Sync و کل دفترچه تلفن پیش از اولین <code>emit</code> تاریخچه تماس	Gate خالی + <code>Future.wait</code> ، رسم فوری از SQLite و اجرای Detached خواندن دستگاه (بخش ۶/۲) – اندازه‌گیری‌شده: ۱/۸ ثانیه تا صفحه پر و رسم‌شده
ابتدا شماره‌ها، یک لحظه بعد اسم‌ها	دفترچه تلفن آن‌سوی کانال پلتفرم است، ولی صندوق و «اخیر» بلافاصله از SQLite رسم می‌شوند	جدول <code>contact_name_cache</code> به فریم اول نام می‌دهد؛ خواندن معتبر پشت سرش اجرا می‌شود و چون State‌ها Equatable‌اند، «هیچ تغییری نبود» یعنی «هیچ بازترسیمی نبود»
کندی و مصرف حافظه با دفترچه تلفن بزرگ	بارگذاری عکس مخاطبین به‌صورت انبوه داخل کش	بارگذاری تنبل عکس به‌ازای شناسه، هرگز داخل کش انبوه
مرتب‌سازی کند دفترچه تلفن	نرمال‌سازی نام داخل تابع مقایسه	الگوی <code>decorate-sort-undecorate</code> – یک کلید به‌ازای هر مخاطب

قواعد پایدار کارایی

- هر فهرست داده‌محور مجازی‌سازی می‌شود؛ خواندن کل جدول ممنوع است.
- هیچ عبارت جستجویی به‌ازای هر مخاطب کامپایل نمی‌شود – یک بار به‌ازای هر فهرست.

- کش فهرست سیم‌کارت‌ها لازم است: چند رفت‌و برگشت Binder دارد و روی مسیر نوتیفیکیشن پیامک (دو بار) و روی هر تغییر وضعیت تماس قرار می‌گیرد.
- در هر بازگشت به برنامه، فقط همگام‌سازی‌های افزایشی مجازند و هر همگام‌سازی یک مالک چرخه‌عمر دارد – نه بیشتر.

۱۸. تست و تضمین کیفیت

راهبرد تست پروژه روی منطق متمرکز است، نه روی پیکسل‌ها: بلاک‌ها، مدل‌های خالص و مخازن داده. برای مخازن، پایگاه‌داده واقعی SQLite در حافظه بالا می‌آید (`sqlite_common_ffi`)، پس تست‌ها روی همان SQL واقعی اجرا می‌شوند نه روی ماک. مجموعه در حال حاضر ۳۹۲ تست در ۳۸ فایل است و در آخرین اجرا کاملاً سبز است (`flutter test` – ۳۹۲ passed).

دسته تست	فایل	نمونه‌ها
بلاک‌ها	۱۰	پیام، تماس، مخاطب، شماره‌گیر، احراز هویت، پیش‌نویس، علاقه‌مندی، تنظیمات، مسدودی، زمان‌بندی
منطق خالص	۱۳	نرمال‌ساز شماره، متن جستجو، الفبای فارسی، تاریخ جلالی، کد یکبارمصرف، قالب پیام، پروتکل فشرده، زمان‌بندی (شامل بردارهای مرجع بذر Jitter)، گفتگوی گروهی
مخازن و همگام‌سازی	۴	مخازن روی SQLite واقعی، حذف تکراری پیامک، نمایه جستجو، سرویس‌های بومی
ویجت	۱۱	صفحه گفتگو، رشد نویسنده پیام، شیت زمان‌بندی، صفحه‌های قالب، و <code>CallUiCoordinator</code> – که Teardown درست Route‌های تماس را در سناریوی Call Waiting قفل می‌کند
ابزار (Instrumentation)	۰	وجود ندارد – تأیید مسیرهای بومی هنوز دستی و روی دستگاه است

سه ناحیه‌ای که بیشترین ارزش تست را دارند

- تست آینه‌ای `Kotlin ↔ Dart` – نرمال‌سازی شماره، منطق زمان‌بندی و پروتکل قالب. ناهماهنگی در این سه، با خطا اعلام نمی‌شود و فقط نتیجه متفاوت می‌دهد.
- منطق حذف تکراری همگام‌سازی – چون تنها چیزی است که بین ابزار سیستم و ابزار برنامه قضاوت می‌کند.
- موتور جستجوی فارسی – پرمصرف‌ترین مسیر و پر از حالت‌های مرزی زبانی.

ابزارهای کیفیت

دستور	کاربرد
<code>flutter analyze</code>	Lint با مجموعه <code>flutter_lints</code>
<code>flutter test</code>	اجرای کامل مجموعه تست
<code>flutter build apk --release</code>	خروجی امضاشده با R8

۱۹. دفترچهٔ تصمیم‌های مهندسی

این جدول برای بازبینی معماری نوشته شده است: هر ردیف یک تصمیم، گزینهٔ رد شده و هزینهٔ پذیرفته شدهٔ آن.

تصمیم	گزینه رد شده	دلیل و هزینه پذیرفته شده
بلاک‌ها سراسری، بدون Provider داخل صفحه	هر صفحه بلاک خودش	وضعیت گفتگوی باز و اسکرول صندوق نباید با ناوبری نابود شود. هزینه: طول عمر همه بلاک‌ها برابر عمر برنامه است.
شماره نرمال شده به عنوان کلید اتصال سراسری	شناسه مخاطب	پیامک و تماس فقط شماره دارند و مخاطب ممکن است اصلاً ذخیره نشده باشد. هزینه: نرمال‌سازی باید در هر دو زبان یکسان بماند.
مخاطبین بدون کپی محلی	آینه کردن دفترچه تلفن در SQLite	آینه دفترچه تلفن یعنی یک مسئله ناهمگامی دائمی. هزینه: هر خواندن باید تنبل و کش شده باشد.
گرفتن نقش کامل پیامک	خواندن فقط از Provider	بدون نقش، همگام‌سازی دوطرفه ممکن نیست و حذف‌ها اثری ندارند. هزینه: مسئولیت ذخیره پیامک ورودی و اجبار به پشتیبانی MMS در سطح Manifest.
گفتگوی گروهی به جای «ارسال انبوه»	یک نویسنده یک بار مصرف که چیزی از خود باقی نمی‌گذارد	بدون MMS هیچ گفتگوی سیستمی‌ای نمی‌تواند چند گیرنده داشته باشد — ولی این فقط تعیین می‌کند پاسخ‌ها کجا می‌آیند، نه اینکه تاریخچه بتواند مشترک باشد. پس: یک گفتگو، یک حباب به‌ازای هر ارسال، <code>N</code> پیامک واقعی، و یک اطلاعیه یک‌بار. هزینه: پاسخ هر عضو در گفتگوی تک‌نفره خودش می‌آید.
یک «شمارنده نسخه» برای هر کش ایستا	اتکا به رفرش بلاک‌ها	دفترچه تلفن و فهرست سیم‌ها هر دو ایستا و همگام درون <code>build</code> خوانده می‌شوند، پس هیچ ویجتی به آن‌ها وابسته نیست و رفرش بلاک فقط سطح‌هایی را درست می‌کند که خود بلاک صاحبشان است. هزینه: هر کش جدید باید شمارنده خودش را هم داشته باشد.
Cache نام مخاطب داخل SQLite، فقط برای فریم اول	رسم ردیف‌ها با شماره و جایگزینی با نام یک لحظه بعد	دفترچه تلفن آن‌سوی یک کانال پلتفرم است و صندوق و «اخیر» بلافاصله از SQLite رسم می‌شوند، پس «شماره حالا، نام بعداً» در هر اجرای برنامه دیده می‌شد و خودش گزارش شد. هزینه: یک جدول Cache که باید فقط با یک خواندن کامل و به‌صورت «جایگزینی کل مجموعه» نوشته شود، وگرنه نام مخاطب حذف‌شده باقی می‌ماند.
آیکن اعلان اختصاصی و برداری، نه آیکن برنامه	استفاده از <code>applicationInfo.icon</code>	آیکن کوچک اعلان یک تصویر نیست: سیستم همه کانال‌ها جز Alpha را دور می‌ریزد و باقی‌مانده را Tint می‌کند، پس آیکن Adaptive تمام‌رنگ و بدون شفافیت به‌صورت یک لکه توپر رسم می‌شد و از نوار وضعیت معلوم نبود کدام برنامه حرفی برای گفتن دارد. هزینه: نگهداری دو Drawable برداری جدا (<code>ic_stat_call</code> و <code>ic_stat_message</code>).
حذف MMS از دامنه	پیاده‌سازی کامل MMS	MMS یک پشته کامل جدا (APN، PDU، رسانه) است و در بازار هدف عملاً بلااستفاده. هزینه: گیرنده MMS یک اجزای بی‌عمل دارد.
نمایه FTS5 ساخته اما غیرفعال	همراه کردن SQLite اختصاصی	اندازه‌گیری شده و رد شده — دو موتور روی یک فایل، ریسک «پیامک از دست‌رفته» دارد (بخش ۱۳).
تحویل زمان‌بندی با ادعای اتمی	یک تحویل‌دهنده واحد	هیچ تحویل‌دهنده واحدی نمی‌تواند هم وقتی برنامه باز است و هم وقتی مرده است کار کند. هزینه: منطق تکراری در دو زبان.

تصمیم	گزینه رد شده	دلیل و هزینه پذیرفته شده
حذف همیشه سراسری + حذف نرم محلی	حذف محلی	حذف محلی تنها، در همگام سازی بعدی برمیگشت. هزینه: Tombstone ها در جدول می مانند.
یک موتور جستجو برای همه جا	تطبیق ساده در هر صفحه	سه صفحه با سه پاسخ متفاوت به یک عبارت، از نبود جستجو بدتر است.
نوتیفیکیشن پیامک تماماً بومی	مسیر Dart	مسیر Dart با پروسه مرده کار نمی کند و داشتن هر دو یعنی نوتیفیکیشن تکراری.
حذف کلیدهای تنظیماتی که چیزی نمی خواندند	نگه داشتن آنها	کلیدی که هیچ جا خوانده نمی شود، به کاربر دروغ می گوید. کلیدهای بازنشسته در اولین بارگذاری پاک می شوند.
بازسازی رابط کاربری بر پایه سامانه طراحی مشترک	استایل به ازای هر صفحه	نقش سطوح و اجزای فهرست مشترک، تنها راه یکدست ماندن ۸ ماژول است.

۲۰. محدودیت‌های شناخته‌شده و نقشه راه

۲۰/۱ محدودیت‌های فعلی – با ذکر صریح دلیل

مورد	وضعیت	توضیح
MMS	خارج از دامنه	گیرنده اجباری نقش وجود دارد ولی پیام را رها می‌کند. افزودن آن یعنی یک پشته کامل جدید.
لایه محلی‌سازی	بدهی	رشته‌های فارسی درون کد نوشته شده‌اند. برای زبان دوم باید به فایل منابع منتقل شوند.
نرمال‌سازی شماره‌های غیرایرانی	ضعیف	قواعد حول پیش‌شماره +98 ساخته شده‌اند؛ شماره خارجی ممکن است به دو گفتگو تقسیم شود.
کرش‌های سمت Kotlin گزارش نمی‌شوند	بدهی	پیامد استفاده از <code>sentry</code> خالص Dart به جای <code>sentry_flutter</code> (بخش ۴).
مسیر VoIP خفته	کد مرده	<code>CallConnectionService</code> به‌عنوان مسیر پشتیبان باقی مانده و در عمل فعال نیست.
تست ابزار (Instrumentation)	ندارد	مسیرهای بومی دستی روی دستگاه تأیید می‌شوند.
نشان «ساعت زنگ» برای پیام زمان‌بندی‌شده	معاوضه	بهای استفاده از <code>setAlarmClock</code> (بخش ۱۴): تا وقتی پیامی برای ۲۴ ساعت آینده در صف باشد، نشان ساعت زنگ سیستم در نوار وضعیت دیده می‌شود. جایگزینش تعویق‌شدن ارسال توسط App Standby است.
حالت تایپی انتخابگر ساعت	محدودیت Flutter	<code>GlobalMaterialLocalizations</code> فارسی فیلد را با ارقام فارسی پر می‌کند و اعتبارسنج خود Flutter با <code>int.tryParse</code> ردش می‌کند – یعنی پنجره پیش از هر تایپی «زمان معتبری وارد کنید» می‌داد. از بیرون قابل رفع نیست، پس انتخابگر فقط حالت عقربه‌ای دارد.
iOS	غیرممکن	iOS جایگزینی برنامه پیامک/تماس را در اختیار توسعه‌دهنده نمی‌گذارد.

اولویت	کار	ارزش
۱	پیامک رمزنگاری شده بین دو کاربر برنامه	درزش از قبل ساخته شده است: پروتکل فشرده قالب، ساختار «سربرگ + فیلدها» را دارد و همان جا می شود بار رمزنگاری شده را نشانند.
۲	استخراج رشته ها به لایه محلی سازی	پیش نیاز هر زبان دوم و شرط ورود به بازارهای دیگر.
۳	تست ابزار برای مسیرهای نقش دار	تنها ناحیه ای که باگ در آن یعنی «پیامک گم شده» یا «تماس بی پاسخ ماندنی».
۴	نرمال سازی شماره بر پایه استاندارد E.164	حل ریشه ای مسئله شماره های بین المللی.
۵	بازبینی تصمیم SQLite اختصاصی	فقط با اندازه گیری روی صندوق بزرگ واقعی و تست همزمانی Dart/Kotlin – نه پیش از آن.
۶	پشتیبان گیری رمزنگاری شده اختیاری	Auto-backup عمداً خاموش است؛ جایگزین امن باید توسط خود برنامه ساخته شود.

جمع بندی مهندسی

آنچه این پروژه را از یک اپلیکیشن معمولی جدا می کند، اندازه رابط کاربری آن نیست؛ بلکه این است که در چند نقطه، *قرارداد آن با سیستم عامل* – و نه با یک سرور – منبع حقیقت است. بیشتر تصمیم های ثبت شده در بخش ۱۹ از همین جا می آیند: جایی که خطا صدا نمی دهد و فقط داده کاربر را بی سروصدا خراب می کند، ارزان ترین کار این است که قاعده را بنویسی و آینه ها را با تست به هم قفل کنی.

۲۱. پیوست – مجوزها و ساخت خروجی

۲۱/۱ مجوزهای اندروید و دلیل هرکدام

گروه	مجوز	کاربرد
پیامک	SEND_SMS	ارسال از نویسنده پیام و کارگر زمان‌بندی
	READ_SMS · RECEIVE_SMS	دریافت زنده و همگام‌سازی آینه‌ای
	RECEIVE_WAP_PUSH · RECEIVE_MMS	اجباری برای نقش پیامک – پیام رها می‌شود
تماس	MANAGE_OWN_CALLS · CALL_PHONE	برقراری تماس و نقش شماره‌گیر
	WRITE_CALL_LOG · READ_CALL_LOG	همگام‌سازی و حذف سراسری تاریخچه
	· READ_PHONE_STATE READ_PHONE_NUMBERS	فهرست سیم‌کارت‌ها و وضعیت شبکه به‌ازای هر سیم
	RECORD_AUDIO	مسیر VoIP (فعلاً خفته)
مخاطبین	WRITE_CONTACTS · READ_CONTACTS	دفترچهٔ تلفن، برچسب‌ها، ادغام تکراری‌ها
سیستم	· POST_NOTIFICATIONS USE_FULL_SCREEN_INTENT	نوتیفیکیشن پیامک و کارت تماس ورودی
	· SCHEDULE_EXACT_ALARM · USE_EXACT_ALARM RECEIVE_BOOT_COMPLETED	تحویل دقیق پیام زمان‌بندی‌شده و بازچینش پس از راه‌اندازی
	VIBRATE · WAKE_LOCK	قفل حسگر نزدیکی و بازخورد لمسی
موقعیت	· ACCESS_FINE_LOCATION ACCESS_COARSE_LOCATION	فقط برای درج مختصات در متن پیام؛ ذخیره یا ردیابی نمی‌شود

جزء	چرا اجباری است
<code>SmsDeliverReceiver</code> (SMS_DELIVER)	بدون آن نقش پیامک اصلاً اعطا نمی‌شود
<code>MmsReceiver</code> (WAP_PUSH_DELIVER)	اجباری؛ عمداً بدون پیاده‌سازی
فیلتر <code>SEND / SENDTO</code> روی <code>MainActivity</code>	اشتراک‌گذاری متن از برنامه‌های دیگر
<code>HeadlessSmsSendService</code> (RESPOND_VIA_MESSAGE)	پاسخ با پیام از صفحه تماس ورودی سیستم
<code>CallInCallService</code>	تنها رابط تماس دستگاه پس از گرفتن نقش شماره‌گیر
ورودی‌های <code><queries></code>	از اندروید ۱۱ به بعد، بدون آن‌ها بخش «برنامه‌های متصل» کاملاً خالی رندر می‌شود

مورد	مقدار / نکته
شناسه بسته	<code>ir.hamrasan.app</code>
امضا	کلید Release اختصاصی؛ تأییدشده در Play Protect
کوچک‌سازی	R8 فعال
معماری هدف	فقط <code>arm64</code> – ساخت با <code>--target-platform android-arm64</code> و حذف استاب‌های <code>abiFilters</code> ۳۲ بیتی از راه قواعد بسته‌بندی، نه
پشتیبان‌گیری	<code>allowBackup=false</code> + قواعد استخراج داده (عمدی، بخش ۱۶)

هم‌رسان – مستند فنی، نسخه ۱/۳

تهیه و تدوین: علی رودی · شهریور ۱۴۰۵

تمام اعداد و ارجاعات این سند مستقیماً از مخزن پروژه استخراج شده‌اند.